Hardware-Oriented SNR-Adaptive Model-Informed LISTA for FPGA-Based Passive-RIS Channel Estimation

A project report submitted in partial fulfilment

of the requirements for the degree of

Bachelor of Technology

in

Electronics and Communication Engineering

by

NAGA RAKESH KUMAR BATTULA (23BEC1006)

JORIGE BALA VENKATA RANGA ROHITH (23BEC1096)

MONISH VENKATA SAI MUNGARA (23BEC1012)



School of Electronics Engineering,

Vellore Institute of Technology, Chennai,

Vandalur–Kelambakkam Road,

Chennai – 600127, India.

October 2026

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

School of Electronics Engineering

Declaration

We hereby declare that the report titled ***Hardware-Oriented SNR-Adaptive Model-Informed LISTA for FPGA-Based Passive-RIS Channel Estimation*** submitted by **Naga Rakesh Kumar Battula** to the School of Electronics Engineering, Vellore Institute of Technology, Chennai, in partial fulfilment of the requirements for the award of **Bachelor of Technology** in **Electronics and Communication Engineering**, is a bonafide record of the work carried out by us under the supervision of ***Dr. P Nirmala***, Associate Professor, School of Electronics Engineering.

We further declare that the work reported in this report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma of this institute or of any other institute or University.

Signature of Student

Naga Rakesh Kumar Battula
Reg. No.: 23BEC1006

Date: 26/10/26.

VIT

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

School of Electronics Engineering

Certificate

This is to certify that the project report titled ***Hardware-Oriented SNR-Adaptive Model-Informed LISTA for FPGA-Based Passive-RIS Channel Estimation*** submitted by ***Naga Rakesh Kumar Battula*** **(23BEC1006)** to Vellore Institute of Technology, Chennai, in partial fulfilment of the requirement for the award of the degree of **Bachelor of Technology** in **Electronics and Communication Engineering** is a bonafide work carried out under my supervision. The project report fulfils the requirements as per the regulations of this University and, in my opinion, meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Supervisor

Signature: VM

Name: Dr. P. Nirmala

Date: 26/10/2026

Head of the Department

Signature: M. Jagannath

Name: Dr. Jagannath M

Date: 27/10/2026

Head of the Department
B.Tech. ECE Programme
School of Electronics Engineering
Vellore Institute of Technology(VIT)
Vandalur - Kelambakkam Road,
Chennai - 600 127.

Examiner 1

Signature:

Name: Dr. T. Thiruvalar

Date: 27/10/26

Examiner 2

Signature:

Name: Dr. Sangeetha R.G

Date: 28/10/26

School of Electronics Engineering
Vellore Institute of Technology
Chennai - 600127, Tamil Nadu, India

(Seal of the School)

Abstract

Passive Reconfigurable Intelligent Surfaces (RIS) present a completely new model for how we can program wireless propagation environments in mmWave and sub-THz frequency bands. However, because RISs lack any type of active radio-frequency (RF) chain, they cannot perform baseband signal processing on the surface. This creates a large problem when trying to estimate channels that are formed as a result of using RISs. For most realistic scenarios where RISs would be deployed, the number of channel coefficients that need to be estimated ($N = 1040$) is much larger than the number of measurements that can be taken using pilots ($M = 256$). Therefore, the channel estimation problem becomes what is known as an "under-determined linear inverse problem." Traditional LS-based estimators will fail completely, while other advanced techniques such as Approximate Message Passing (AMP) and Vector AMP (VAMP) will usually fail to converge because of the Kronecker-structured sensing matrices created by the physical layout of the elements on the RIS.

To circumvent these fundamental barriers, the proposed project develops an entire, end-to-end, hardware-oriented channel estimation system that combines an Early Noise Estimation (ENE) network, an 8-layer Model-Informed Learned Iterative Shrinkage-Thresholding Algorithm (LISTA) core, and a lightweight 3-layer Convolutional Neural Network (CNN) denoiser. With Rademacher phase configurations, the developed design does away with the need for explicit sensing matrix multiplications, thereby releasing all Digital Signal Processing (DSP) slices from projection operations. The system is entirely implemented in C++ High-Level Synthesis (HLS) utilizing 16-bit fixed-point arithmetic (`ap_fixed`) and is designed to operate on the AMD-Xilinx Zynq UltraScale+ ZCU104 MPSoC operating at 100 MHz.

An experimental validation conducted over the cable using board level testing has demonstrated a normalized mean square error (NMSE) of -14.93 dB at an operational Signal-to-Noise Ratio (SNR) of 20 dB, representing an exceptionally small implementation loss of only 0.065 dB compared to IEEE floating point simulation. The proposed pipeline also exhibits a deterministic execution latency of 23.27 ms per channel estimate, resulting in an industrial pass rate of 93.3% (≤ -10 dB), while consuming only 1.3 W of total on-chip power. Therefore, this work provides a mathematically sound, low power consumption framework for implementing real-time edge processing within the context of 6G cellular communication networks.

Acknowledgements

We wish to express our heartfelt gratitude and profound sense of respect to our project supervisor, **Dr. P Nirmala** , Associate Professor, School of Electronics Engineering (SENSE), Vellore Institute of Technology, Chennai. Her deep academic insight, meticulous guidance, constant encouragement, and constructive critique throughout the course of this work were pivotal in navigating the mathematical formulations and FPGA implementation challenges of this thesis.

We sincerely express our deep gratitude to **Dr. Ravi Sankar A**, Dean, School of Electronics Engineering; **Dr. Mohanaprasad K**, Associate Dean (Academics); and **Dr. Chitra K**, Associate Dean (Research), for their administrative leadership, fostering an inspiring research environment, and facilitating access to the advanced computing and embedded systems laboratory facilities.

We extend our sincere thanks to **Dr. Jagannath M**, Head of the Department, for his constant administrative support, academic motivation, and valuable suggestions during the reviews of this project.

We also take this opportunity to convey our gratitude to all faculty members, technical officers, and laboratory staff of the School of Electronics Engineering, whose knowledge, pedagogical support, and timely assistance enriched our undergraduate learning experience throughout our B.Tech tenure.

Lastly, we express our boundless gratitude to our parents, family members, and friends. Their unwavering moral encouragement, patient sacrifices, and steadfast belief in our endeavours provided the enduring foundation upon which this project was completed.

Codes and Standards

The research, system modeling, hardware design, and physical validation described in this project strictly comply with international telecommunication standards, digital hardware design specifications, and embedded system verification protocols as outlined below:

1. **Wireless Communications & Spatial Propagation Standards**
    - **3GPP TR 38.901 – Study on Channel Model for Frequencies from 0.5 to 100 GHz**
      Governs the 3D spatial channel model (SCM), clustered delay line (CDL) geometries, angular spreads (AoA/AoD), and subpath distributions employed for the base station (BS), passive RIS, and user equipment (UE) propagation links.
    - **IEEE 802.11ay / IEEE 802.11ad – mmWave Wireless LAN Standards**
      Specifies physical-layer frame structures, beamforming training protocols, and codebook quantization metrics applicable to high-frequency directional antenna array systems.
    - **ITU-R M.2410-0 – Minimum Requirements for IMT-2020 Radio Interfaces**
      Defines target spectral efficiency, latency tolerances, and energy efficiency constraints guiding 5G-Advanced and 6G infrastructure development.
2. **Digital Arithmetic, Hardware Description & Bus Interface Standards**
    - **IEEE 754-2019 – Standard for Floating-Point Arithmetic**
      Establishes the double-precision (FP64) and single-precision (FP32) numerical reference models used to evaluate algorithm convergence and quantify fixed-point implementation error.
    - **ISO/IEC 14882:2017 (C++17) / Vivado HLS Synthesis Specifications**
      Defines syntax and semantic constraints for synthesizable template metaprogramming, arbitrary-precision integer/fixed-point definitions (`ap fixed<16,8>`), and pragma pipelining.
    - **ARM AMBA AXI4 / AXI4-Stream Protocol Specification (IHI 0022E)**
      Governs memory-mapped and streaming handshakes between the ARM Cortex-A53 Processing System (PS) and the FPGA Programmable Logic (PL) via Direct Memory Access (DMA).
    - **IEEE 1800-2017 – SystemVerilog Hardware Design and Verification**
      Provides the standard language and structural verification rules applied during RTL generation, co-simulation, and timing closure.
3. **Electronic Test, Instrumentation & Verification Standards**
    - **IEEE 1149.1-2013 – Standard Test Access Port and Boundary-Scan (JTAG)**
      Utilized for in-circuit hardware programming, bitstream flashing, and non-intrusive register state inspection on the AMD-Xilinx ZCU104 evaluation platform.

- **IEC 61508 / ISO 26262 – Functional Safety and Verification Metrics** Informs our automated hardware-in-the-loop regression methodology across 1,000 independent channel test vectors to ensure deterministic bounding of numerical failure tails.

Mapping of Sustainable Development Goals (SDGs)

Project Title: Hardware-Oriented SNR-Adaptive Model-Informed LISTA for FPGA-Based Passive-RIS Channel Estimation

Project Title	Mapped SDGs	Justification	Example Impact
Hardware-Oriented SNR-Adaptive Model-Informed LISTA for FPGA-Based Passive-RIS Channel Estimation	**SDG 9** – Industry, Innovation and Infrastructure	The research develops novel, energy-efficient deep-unrolled neural architectures and specialized FPGA hardware cores, advancing telecommunications infrastructure for next-generation 6G mobile networks.	Enables ultra-low-latency channel reconstruction in edge radio base stations with deterministic hardware execution.
	SDG 11 – Sustainable Cities and Communities	Deploying passive metasurfaces extends reliable, high-speed wireless connectivity into dense urban canyons and indoor coverage blind spots without installing massive active cellular towers.	Eliminates urban dead zones and enhances municipal connectivity for smart transit, public safety, and civic IoT networks.
	SDG 12 – Responsible Consumption and Production	Passive RIS operate without power-hungry RF power amplifiers, mixers, or DAC/ADC chains, slashing raw electronic hardware requirements and reducing toxic electronic waste (e-waste).	Replaces costly active relay towers with recyclable electromagnetic metasurfaces having minimal lifecycle footprint.

Continued on next page

(Continued from previous page)

Project Title	Mapped SDGs	Justification	Example Impact
	SDG 7 – Affordable and Clean Energy	The developed 16-bit fixed-point core achieves real-time inference within a frugal 1.3 W on-chip power budget, drastically shrinking the carbon footprint of telecom network operations.	Cuts base-station operational energy expenditure, directly supporting grid decarbonization and green communications.

Contents

Declaration i

Certificate ii

Abstract iii

Acknowledgements iv

Codes and Standards v

Mapping of Sustainable Development Goals (SDGs) vii

List of Figures xii

List of Tables xiii

List of Acronyms xiv

1 Introduction 1

1.1 Background and Motivation 1

1.1.1 Fundamentals of Reconfigurable Intelligent Surfaces 3

1.1.2 Propagation Challenges at Millimeter-Wave Frequencies 4

1.1.3 Cascaded Channel Representation and Dimensionality 5

1.1.4 Pilot Overhead and Channel Coherence 6

1.1.5 Angular-Domain Sparsity of Wireless Channels 7

1.1.6 Compressive Sensing for Channel Reconstruction 8

1.1.7 Hardware-Aware Channel Estimation 9

1.2 Research Challenges 10

1.2.1 Algorithmic Behaviour Under Structured Sensing 11

1.2.2 Adaptation to Changing Signal-to-Noise Ratios 11

1.2.3 Computational Complexity and Hardware Constraints 12

1.3 Research Gap and Motivation 12

1.4 Problem Statement 14

1.5 Objectives 16

1.6 Scope of the Work 18

1.6.1 Wireless System Model 18
1.6.2 Pilot Budget and Observation Model 18
1.6.3 Channel Representation and Reconstruction Architecture 19
1.6.4 Hardware Platform and Numerical Representation 19
1.6.5 Evaluation Conditions 20
1.6.6 Exclusions and Limitations 20
1.7 Intended Contributions 20
1.8 Organisation of the Report 21

2 Literature Survey 23
2.1 Review of Prior Work 23
2.1.1 Passive RIS Cascaded Channel Estimation Paradigms 23
2.1.2 Compressive Sensing and Message-Passing Solvers 24
2.1.3 Deep Unrolling and Learned Iterative Shrinkage 25
2.1.4 Hardware Accelerators and FPGA Implementations 26
2.1.5 Positioning Against Contemporary Art 27
2.2 Gaps Identified 27

3 Methodology 29
3.1 System Architecture and Mathematical Model 29
3.1.1 Cascaded Channel Formulation 29
3.1.2 Virtual Angular Domain Representation 31
3.1.3 Rademacher Structured Sensing Matrix 31
3.2 Deep Unrolled Neural Network Architecture 32
3.2.1 Overall System Dataflow 32
3.2.2 Early Noise Estimation (ENE) Network 32
3.2.3 Model-Informed LISTA Core 34
3.2.4 Datapath of One Unrolled Core Layer 35
3.2.5 Lightweight 3-Layer CNN Denoiser 35
3.3 FPGA Architecture and Fixed-Point Hardware Design 36
3.3.1 Target Platform and System Partitioning 36
3.3.2 16-Bit Fixed-Point Arithmetic and Precision Analysis 37
3.3.2.1 Mathematical Proof of Bounded Quantization Error Accumulation 37
3.3.3 AXI DMA Interfacing and Hardware-in-the-Loop Protocol 38
3.4 Experimental Verification Framework 38

4 Results and Discussion 40
4.1 Software Simulation and Algorithmic Results 40
4.1.1 NMSE versus SNR Performance Benchmark 40
4.1.2 Architectural Ablation Analysis 41
4.1.3 In-Depth Investigation of Message-Passing Divergence 42
4.2 Hardware Synthesis and Implementation Results 43
4.2.1 Resource Utilization Breakdown on AMD ZCU104 43
4.2.2 Latency and Execution Timing Breakdown 44
4.3 On-Board Physical Validation and Statistical Evaluation 45
4.3.1 Bit-Accurate Verification and Quantization Loss 45
4.3.2 1,000-Vector Statistical Scatter and Pass-Rate Analysis 45
4.3.3 Root-Cause Analysis of Tail Failures 46

4.4 Discussion and Positioning 47
4.4.1 Comparative Evaluation Against Prior Art 47
4.4.2 Limitations and Practical Considerations 48
4.5 Conclusion 48

A Appendix **49**
A.1 Synthesizable Vivado HLS C++ Core Implementation 49
A.2 Detailed Mathematical Proof of Fixed-Point Error Boundedness 52
A.2.1 Properties of Sub-Operators 52
A.2.2 Error Propagation Recurrence 53
A.3 Embedded ARM Cortex-A53 Linux AXI DMA Driver 54

Bibliography **56**

Biodata **58**

List of Figures

3.1 System dataflow and hardware-software partitioning between Processing System (PS) and Programmable Logic (PL) on the ZCU104 MPSoC. 33
3.2 Microarchitecture datapath of one unrolled core layer (blue: sensing-operator and state registers; orange: transform-domain FFT/IFFT and thresholding blocks). 35

4.1 Normalized Mean Squared Error (NMSE) versus SNR comparison between the proposed Model-Informed LISTA framework and baseline estimators ($N = 1040, M = 256$). 42
4.2 Empirical NMSE distribution across 1,000 independent channel test realizations on the AMD ZCU104 FPGA at SNR = 20 dB (933 Passed ≤ -10 dB, 67 Failed > -10 dB). 46

List of Tables

2.1 Critical Comparison of Prior Art in RIS Channel Estimation and Deep Unrolling Hardware 27

4.1 Ablation Study of Architectural Components at SNR = 20 dB 42
4.2 Post-Implementation FPGA Resource Utilization Breakdown on AMD ZCU104 at 100 MHz 44
4.3 Stage-by-Stage Hardware Execution Latency Profile at 100 MHz 44
4.4 Numerical Verification of Floating-Point Simulation vs. Fixed-Point FPGA Board 45
4.5 Performance Comparison with State-of-the-Art FPGA Channel Estimators 47

List of Acronyms

Acronym	Full Form
3GPP	3rd Generation Partnership Project
ADC	Analog-to-Digital Converter
AMP	Approximate Message Passing
AoA	Angle of Arrival
AoD	Angle of Departure
ASIC	Application-Specific Integrated Circuit
AWGN	Additive White Gaussian Noise
AXI	Advanced eXtensible Interface
BRAM	Block Random Access Memory
BS	Base Station
CDF	Cumulative Distribution Function
CDL	Clustered Delay Line
CNN	Convolutional Neural Network
CS	Compressive Sensing
CSI	Channel State Information
DAC	Digital-to-Analog Converter
DFT	Discrete Fourier Transform
DMA	Direct Memory Access
DS-OMP	Distributed Sparsity Orthogonal Matching Pursuit
DSP	Digital Signal Processing
ENE	Early Noise Estimation
FF	Flip-Flop
FFT	Fast Fourier Transform
FISTA	Fast Iterative Shrinkage-Thresholding Algorithm
FPGA	Field-Programmable Gate Array
GPU	Graphics Processing Unit
HLS	High-Level Synthesis
IFFT	Inverse Fast Fourier Transform
IoT	Internet of Things
ISTA	Iterative Shrinkage-Thresholding Algorithm
JTAG	Joint Test Action Group
LISTA	Learned Iterative Shrinkage-Thresholding Algorithm
LoS	Line-of-Sight
LS	Least Squares
LUT	Look-Up Table
MIMO	Multiple-Input Multiple-Output
MPSoC	Multi-Processor System-on-Chip
MSE	Mean Squared Error

(Continued from previous page)

Acronym	Full Form
NLoS	Non-Line-of-Sight
NMSE	Normalized Mean Squared Error
OMP	Orthogonal Matching Pursuit
PL	Programmable Logic
PS	Processing System
RF	Radio Frequency
RIS	Reconfigurable Intelligent Surface
RTL	Register Transfer Level
SDG	Sustainable Development Goal
SNR	Signal-to-Noise Ratio
SVD	Singular Value Decomposition
UE	User Equipment
ULA	Uniform Linear Array
URAM	Ultra Random Access Memory
VAMP	Vector Approximate Message Passing

Chapter 1

Introduction

1.1 Background and Motivation

The continued advancement of wireless communications is fueled by the need for greater data rates, enhanced spectral efficiency, reduced communication latency, and increased reliability in increasingly challenging propagation environments. While 5G wireless networks provided a significant improvement in mobile broadband and large scale antenna deployments, it is anticipated that 6G communication networks will be able to provide even greater levels of performance than 5G through the use of advanced antenna technology, intelligent radio environments, artificial intelligence and higher frequency spectrums [1, 2]. Consequently, there is a clear motivation for the development of physical layer technologies that can improve the propagation characteristics of the channel, as opposed to solely relying on traditional transmitter and receiver processing.

An RIS (Reconfigurable Intelligent Surface) is one technology that has been studied for use in future wireless communication systems. An RIS is made up of a number of engineered electromagnetic components whose properties are able to be changed so that they may affect how incident electromagnetic waves travel. Each component on the RIS may have its own reflective properties modified, thus allowing an RIS to change the phase and possibly the amplitude of the reflected electromagnetic wave. Therefore, an RIS is capable of changing certain aspects of the wireless communication channel via programmable electromagnetic components.

A conventional active relay has a dedicated radio frequency (RF) transmission chain for each reflecting element; however, a passive RIS may use its elements to alter the electromagnetic field that impinges upon them by changing how they scatter the incident wave. As a result, the RF hardware requirements associated with large intelligent

surfaces may be reduced. However, even though the elements of a passive RIS do not need a dedicated RF transmission chain, the elements will still need some type of control circuitry, configuration interface and possibly bias/switch power. Therefore, the overall energy efficiency of a passive RIS deployment will depend on the entire deployment of the RIS and not just because it is a passive reflecting element.

RIS technology has special relevance to mmWave communication due to its large bandwidth capabilities, and the potential to use small antennas in an array configuration which allows for high capacity wireless link. However, mmWave propagation is limited by; blockage of the signal, directivity in the propagation of the signal, penetration loss into materials, and the geometry of the surrounding environment. For example, in urban areas, buildings, vehicles and other objects could block the direct propagation path from the transmitting device to the receiving device. A properly designed RIS can create a secondary propagation path for the signal through reflection, providing an improved connection under certain geometric and channel conditions.

The ability of a RIS assisted communications system to function successfully will be dependent on more than just the physical surface. It will also depend upon the ability of the transmitter, RIS, and receiver to have enough knowledge about the actual wireless channel so that they can perform coherent detection, beamforming and reliably transmit data. Therefore, the ability to accurately estimate the wireless channel is critical in order to achieve the advantages that exist in a RIS assisted wireless systems.

This problem is especially challenging if the surface is composed of many elements. Increasing the number of elements allows for greater spatial control; however, it also increases the dimension of the channel estimation problem. Traditional methods of channel estimation could require a large quantity of pilot observations, thus resulting in an increased training overhead and fewer available resources for transmitting payload. Therefore, there is motivation to explore alternative channel estimation methods that are structured and capable of estimating high-dimensional channels based upon a small quantity of observations.

The computation required for the RIS channel estimation algorithm must be taken into account as well as how accurate it is at estimating. While some algorithms do very well in floating-point software, they will need a lot of computational resource, memory bandwidth, and time to run on embedded hardware. Therefore, the design of an effective RIS channel estimator should take into account all of these factors; propagation models, reconstruction algorithms, noise environments, numerical precision, and the specific hardware platform that it is designed for.

1.1.1 Fundamentals of Reconfigurable Intelligent Surfaces

In general, an RIS can be represented as a two-dimensional array of N_{ris} controllable electromagnetic components. Each component modifies the incident electromagnetic field with its available reflection configurations. The overall response based upon the phase relationship of the light reflected from various areas of the surface will determine the components of the wave which arrive at the receiver.

The mathematical description for this would be a diagonal form of the RIS configuration.

reflection matrix:

$$\boldsymbol{\Theta} = \text{diag}\left(\beta_1 e^{j\phi_1}, \beta_2 e^{j\phi_2}, \ldots, \beta_{N_{ris}} e^{j\phi_{N_{ris}}}\right), \tag{1.1}$$

In this equation, β_n represents the reflection coefficient of each individual element and ϕ_n represents the adjustable reflection phase for each element. This parameter $j = \sqrt{-1}$ is the imaginary number.

The ideal unit-modulus reflection model assumes that the amplitudes are equal to one ($\beta_n = 1$). However, in practice, many surfaces will have loss of amplitude, amplitude-phase coupling, and only a limited number of possible phases that can be used. Thus, while the ideal model can be used for analysis mathematically, it may need to be modified in order to accurately represent a particular type of physical RIS.

The received signal will depend on the electromagnetic contribution from both the direct propagation path and the RIS assisted path. In a narrow band, single user, uplink system, a simple baseband representation is:

$$\mathbf{y} = (\mathbf{h}_d + \mathbf{G}\boldsymbol{\Theta}\mathbf{h}_{ru})\,x + \mathbf{n}, \tag{1.2}$$

The variable $\mathbf{h}d \in \mathbb{C}^{N_r}$ is used to represent the direct UE-to-BS channel; the variable $\mathbf{h}ru \in \mathbb{C}^{N_{ris}}$ is used to represent the UE-to-RIS channel; the variable $\mathbf{G} \in \mathbb{C}^{N_r \times N_{ris}}$ is used to represent the RIS-to-BS channel. The variable x represents either a pilot or a data symbol that was transmitted by the UE, while the variable $\mathbf{n} \in \mathbb{C}^{N_r}$ represents the additive noise at the BS receiver.

Equation (1.2) shows how the total channel is determined by both of the propagation channels and the RIS setup. In other words, the receiver is seeing the combination of all of the effects from both propagation channels and the RIS, as opposed to being able to measure each of the propagation coefficients individually.

An RIS is able to affect how the different reflected parts of a signal are phased relative to each other by controlling each part's reflection phase. In some situations (e.g., when the propagation medium allows), the different reflected parts can combine constructively

at a specific point in space where a user would like to receive the signal. The amount of additional signal strength received because of the RIS depends upon several factors including; the channel geometry, the efficiency of the RIS, the physical size of the RIS array, and the accuracy with which the RIS has been set up. Therefore, it is important to remember that the primary function of an RIS is to allow for control of certain propagation paths and not as a complete replacement for a direct line-of-sight connection.

1.1.2 Propagation Challenges at Millimeter-Wave Frequencies

Although mmWave systems offer significant bandwidth, there are several propagation-related problems that create difficulties in providing a stable wireless link. The use of higher carrier frequencies allows for the design of smaller antenna arrays that have a large number of individual elements in a small physical space. These arrays can be designed to provide directional transmission/reception capabilities so that the array will direct its energy toward specific spatial locations.

However, how well directional mmWave links perform will depend on if there are appropriate propagation paths available and if the system has accurate spatial channel data. There are many factors that could affect the quality of the received signal such as; blockages, antenna orientation, material types, and the overall environment in which the system is being operated. If the direct path is blocked, then it is possible for the system to use reflected signals as a way to find other propagation paths, however, whether or not these reflected signals can be used will depend on the path losses associated with them and the geometric characteristics of the reflection.

An RIS (Reconfigurable Intelligent Surface) can provide assistance in such environments, through the control of reflection characteristics of a surface that is located on a potential propagation path. The settings of an RIS may be chosen to modify the phase relationships of the reflected fields, and thus enhance the link quality if the environment allows for this.

However, the system must obtain enough channel information to find the appropriate configuration and process the received signals. Therefore, there is an interaction between propagation control and channel estimation. That is, the success of the intelligent surface configuration depends on the quality of the channel information obtained by both the receiver and the controller.

Due to the vast amount of reflecting components and the directional qualities of mmWave propagation, it is critical to develop methods for efficiently estimating. As such, we have

been motivated to utilize channel models that incorporate both antenna-array geometry and the nearly sparse quality of propagation in the angular domain.

1.1.3 Cascaded Channel Representation and Dimensionality

For the system considered in this thesis, the RIS-assisted effective channel is expressed as

$$\mathbf{h}_{\text{eff}} = \mathbf{h}_{\text{d}} + \mathbf{G}\Theta\mathbf{h}_{\text{ru}}. \tag{1.3}$$

The dimension of each individual term determines the dimension of the channel estimation problem. The direct channel is represented by N_r complex coefficients; however, the RIS-assisted contribution is dependent upon the $N_r \times N_{ris}$ matrix $\mathbf{G}$, and the $N_{ris} \times 1$ vector $\mathbf{h}_{ru}$.

For a given configuration of the RIS, the effective channel $\mathbf{h}_{\text{eff}}$ has N_r complex components. When we consider the estimation of each coefficient of the cascaded channel for each possible configuration of the RIS, we are dealing with a much larger number of unknown variables.

Defining the cascaded channel matrix as

$$\mathbf{H}_{\text{c}} = \mathbf{G}\,\text{diag}(\mathbf{h}_{\text{ru}}), \tag{1.4}$$

gives $\mathbf{H}_{\text{c}} \in \mathbb{C}^{N_r \times N_{\text{ris}}}$. The number of coefficients in this matrix is

$$N_{\text{c}} = N_r N_{\text{ris}} = 16 \times 64 = 1024. \tag{1.5}$$

When the direct BS-to-UE channel is also included in the unknown parameter vector, it contributes an additional $N_r = 16$ complex coefficients. The resulting augmented representation therefore contains

$$N = N_r N_{\text{ris}} + N_r = 16 \times 64 + 16 = 1040 \tag{1.6}$$

complex coefficients.

The distinction between the increased dimension of parameters and the number of entries in the effective channels for each configuration of an RIS is important. The increased dimension of parameters (the augmented vector) represents all the unknown parameters being reconstructed, while the effective channel represents how those unknown parameters combine to create what we observe at the receiver.

In addition to being a way of forming a linear estimation model, the use of the cascade representation will depend upon the used parametrization. The entries in the cascaded channel are formed by multiplying the corresponding underlying propagation coefficients and may not be independent. Also, the recovery of the cascaded channel does not necessarily mean that the separate UE-to-RIS and RIS-to-BS channels were determined uniquely.

1.1.4 Pilot Overhead and Channel Coherence

In order for channel estimation to occur, there must be a sufficient number of training observations to allow the receiver to identify the unknown propagation coefficients. The ability to uniquely recover the unknown vector is determined by both the number of training observations and how they are arranged within the conventional unconstrained linear estimation problem.

For a linear system with N arbitrary unknown coefficients, usually at least N independent scalar measurements are needed to uniquely recover the system, unless there are other structural assumptions made about the system. A sensing matrix that has fewer than N rows will not be able to have full column rank. Therefore, unless you have some sort of additional information, an underdetermined system usually has many possible solutions.

The above requirement does not apply to all pilot requirements in various channel estimation techniques. For example, sparse recovery techniques may be able to estimate channels with fewer measurements if there are certain structural properties inherent within the channel and if the sensing matrix used in the measurement process contains enough information to allow for stable estimation.

In RIS-assisted systems, the receiver must consider the different types of RIS configurations that were utilized during the pilot transmission process. Each type of RIS configuration can result in varying measurements of the unknown cascaded channel. When the various RIS configurations result in measurements that are highly correlated with each other, then regardless of the total amount of data collected (measurements), the amount of usable data that can be used to reconstruct the unknown channel will be restricted.

In addition to being evaluated based upon how much pilot overhead will be generated by the channel coherence interval, the channel coherence interval is defined as the time-frequency area in which the channel can be viewed as stable enough to support a specific type of channel estimation. This is due to many different variables including

user movement, carrier frequency, propagation characteristics of the channel, and system bandwidth.

When a large part of the time that can be used for transmission of data is taken up by estimating, then there are fewer options available to transmit user data. Therefore, reducing the number of pilot measurements used in order to keep the estimation error down is a very important goal in actual wireless systems development.

1.1.5 Angular-Domain Sparsity of Wireless Channels

A common feature of most mmWave propagation environments is that only a few dominant propagation paths contribute to a significant amount of the total received signal power. The dominant paths are often caused by direct propagation and/or reflection from a few different geometrically separated directions.

Although actual environments contain diffuse scatterers and smaller amounts of propagated energy, the channel can typically be represented by a relatively small number of large angular components. This characteristic provides the motivation for representing the channel in a coordinate system based upon spatial directions, as opposed to directly manipulating every single antenna-domain coefficient.

A plane wave response of a uniform linear array with N_r antenna elements may be expressed as a steering vector

$$\mathbf{a}(\theta) = \sqrt{\frac{1}{N_r}}\left[1 \quad e^{j2\pi(d/\lambda)\sin\vartheta} \quad \dots \quad e^{j2\pi(d/\lambda)(N_r-1)\sin\vartheta}\right]^T, \tag{1.7}$$

where d represents the antenna spacing, λ represents the carrier wavelength, and θ represents the angle from the selected array convention.

The channel is estimated by summing the product of each steering vector representing the strongest propagation path and its weight. When the channel in the antenna domain is converted into an angular representation, most of the energy will be concentrated into a limited amount of coefficients.

The 8 × 8 uniform planar array used as the RIS can have its variation represented in two dimensions. This representation provides a good way to describe how the angles in the channel are structured.

The amount of sparsity will depend upon the characteristics of the environment where the signal propagates, the geometry of the array, the level of angular resolution, and whether or not the direction(s) of the actual path(s) correspond to one (or more) of the discrete transform grids. If a physical path does not coincide with a grid direction, then

its energy may diffuse into multiple transform coefficients. Therefore, the channel can be considered to be somewhat sparse, as opposed to strictly sparse.

In addition, the use of this angular-domain structure allows for a reduction in the overall complexity associated with channel estimation. Therefore, the performance of the proposed reconstruction framework will need to be assessed by utilizing realistic channel realizations as opposed to assuming that each of the actual channels has the exact same sparse representation.

1.1.6 Compressive Sensing for Channel Reconstruction

Compressive sensing is a mathematical tool that allows us to find structured signals using fewer linear measurements than there are coefficients to be determined. The goal with RIS assisted channel estimation is to take advantage of the approximate sparsity of the channel in some form of representation rather than trying to determine each individual coefficient.

Consider the observation model

$$\mathbf{y} = \mathbf{\Phi h} + \mathbf{n}, \tag{1.8}$$

where $\mathbf{y} \in \mathbb{C}^M$ denotes the observation vector, $\mathbf{\Phi} \in \mathbb{C}^{M \times N}$ is the sensing matrix, $\mathbf{h} \in \mathbb{C}^N$ is the unknown channel parameter vector, and $\mathbf{n}$ denotes measurement noise.

When $M < N$, the system is underdetermined. A sparse reconstruction method addresses this problem by seeking a solution containing relatively few significant coefficients. One common formulation is

$$\hat{\mathbf{h}} = \arg\min_{\mathbf{h}} \frac{1}{2}\|\mathbf{y} - \mathbf{\Phi h}\|_2^2 + \lambda \|\mathbf{h}\|_1, \tag{1.9}$$

the parameter λ determines the balance of how consistent the measurements will be with respect to consistency and how sparse the solution will be. The ℓ_1 norm of a complex valued vector is simply the sum of all elements absolute values.

Equation (1.9) includes two terms. One term will discourage the candidate channel vector from making predictions that do not agree with the observed measurements. The other term will promote solutions that have only a few large coefficients.

The selection of an appropriate value for the regularization parameter is essential to achieve a balance between the accuracy of the reconstructed image and its stability against noise. While a high value of the regularization parameter will reduce noise and suppress irrelevant coefficients, it will also eliminate true weak propagation paths. On the other hand, if the regularization parameter is too small, then some of the coefficients

that are purely the result of noise will remain and this will decrease the quality of the reconstructed image.

Compressive sensing is not guaranteed to recover a signal from an arbitrary under-determined system. To recover a signal using compressive sensing, the sensing matrix must contain enough information about the signal's structure, and the communication channel must meet requirements compatible with the reconstruction algorithm. Therefore, there is motivation to investigate the design of sensing structures and reconstruction algorithms for use in channels assisted by reconfigurable intelligent surfaces (RIS).

1.1.7 Hardware-Aware Channel Estimation

Deployment of a trained channel estimation model is much larger than simply converting the trained model's neural network into a hardware description language. To find which operations in the model's computational graph take the longest to run, are the most resource intensive (i.e. use up the most logic), and/or have the highest data dependency requirements, it will need to be analyzed.

Iterative reconstruction networks require multiple iterations of computing the measurement operator and its adjoint. Since these computations are typically done using general dense matrix multiplication, they can be very computationally expensive. The expense of the computations depends on several factors including: number of iterations, precision used for arithmetic operations, amount of data that is reused throughout the process, how memory is organized within the system, and level of parallelization available.

Field-programmable gate arrays (FPGAs) provide the ability to run multiple operations in parallel, as well as use pipelining. With independent arithmetic operations, it is possible to schedule these operations to occur at the same time, and with pipeline stages, it is possible to continue processing data even after some of the earlier steps have completed, so that you do not have to wait until all of the current step's operations are complete prior to accepting the next item of data.

However, the benefits are dependent on the number of logic resources available, the clock speed, the bandwidth of the memory, how dependent the data is, and how many parallel operations can be performed with the design. It is possible for a design to reduce the number of arithmetic operations but still consume more time or energy than another design when implemented in a specific type of FPGA.

Fixed-point arithmetic is an alternative method to minimize hardware expenses. In contrast to floating-point representation, fixed-point representations allow the designer to assign a specific amount of bit space to each of the integer and fractional parts of a numeric

value. The advantages of fixed-point representations are simpler arithmetic operations and less required memory. However, there are disadvantages such as quantization errors and rounding errors that occur during the conversion from floating-point representation to fixed-point representation. Furthermore, accumulation may need more bit space for integers in order to avoid overflow.

The first implementation will use ap fixed as its baseline numeric format. The baseline numeric format has sixteen total bits and eight integer bits (which include the sign bit), based upon the traditional AMD Vitis HLS syntax. All of the remaining bits are for representing the fractional part. Depending on the need for multiplication, accumulation, rounding, and/or saturation; intermediate results may require a larger numeric format.

Structured sensing operators offer a possible means to reduce the expense of performing projection operations. An operator which is implemented using only sign changes, permutations, and additions will have fewer requirements for multi-purpose multiplication than most operators. It should be noted however, that a matrix composed solely of $+1$ and -1 entries is not guaranteed to have an easily implementable form simply due to the fact that all of its components are either signs. A appropriate factorization or structured operator definition must be developed and verified.

The same is true for the FFT/IFFT block, which can perform some type of operation within the transform-domain effectively. However, the amount of hardware resources required by these blocks will depend on the architecture, numerical formats, and synthesis configurations used. Therefore, the hardware resources required must be determined through the use of synthesis and implementation reports.

A hardware aware approach to design will be motivated by the need to consider the representation used for sensing, the algorithm used for reconstructing the image, the number of bits of precision to use when performing calculations, the layout of memory within the system, and how data moves through the system.

1.2 Research Challenges

While there may be advantages to using a sparse channel estimate and/or hardware acceleration, there are still some issues that need to be resolved prior to the use of the resulting estimator within an embedded environment. The issues studied during this research are listed below.

1.2.1 Algorithmic Behaviour Under Structured Sensing

Approximate message passing (AMP) was created as a iterative algorithm that could be applied to high dimensional inference issues. AMP has shown some positive results with regards to reconstruction performance under certain random matrix and large system conditions. Although, AMP does not necessarily perform well on all sensing matrices.

The use of RIS for channel estimation produces sensing matrices that possess significant structure and correlation. These matrices can inherit properties from antenna steering vectors, RIS phase patterns, and separable spatial transformations. Therefore, these matrices can vary greatly from the independent, identically distributed (i.i.d.) Gaussian ensemble assumed in many classical AMP analysis.

Under such circumstances, convergence can be affected by the conditioning of matrices, how measurements are scaled and other parameters of an algorithm. Vector AMP (and other similar methods) are designed to alter the process of inference in order to allow for a wide range of different types of matrix structures; however, they will always be dependent upon both what is assumed about them, as well as how they were implemented.

The evaluation of classical iterative methods as reference baseline is motivated by these findings. The convergence properties, accuracy of reconstruction and computational cost should all be evaluated using the same sensing model and channel realization which was used for the evaluation of the proposed estimator.

1.2.2 Adaptation to Changing Signal-to-Noise Ratios

The signal to noise ratio (SNR) represents the relationship between the level of the received signal and the level of measurement noise, based on a given normalization. The actual SNR will depend upon several factors such as propagation loss, interference, receiver noise, transmit power and channel characteristics in real-world wireless environments.

The way that the regularization and threshold parameters are set will determine how well you preserve true channel components and eliminate noise. If a threshold works well at one SNR, it might not work well at another. If you set your threshold too high, you could lose some of your weaker paths; however, if you set your threshold too low, you may keep components that were created by noise.

In learned iterative shrinkage-thresholding algorithms (LISTA) the reconstruction parameters can be learned from the available training data. However, if a model has been trained within a limited distribution of noise conditions, it will most likely have poorer generalization capabilities over the entire operational SNR range.

This justifies adding an Early Noise Estimation (ENE) module that will allow us to assess the present noise situation and use this assessment to adjust our reconstruction settings. We must verify how well it works throughout the full range of SNRs we want to cover, even under conditions that were never used to train the model.

1.2.3 Computational Complexity and Hardware Constraints

The computational requirements of a channel estimator are dependent upon the number of calculations that occur within each of the reconstruction steps. For example, when there is a need to perform numerous calculations with dense, complex-valued matrix-vector products, it will typically require many arithmetic units and/or a large amount of memory to store intermediate results. If the calculation is done multiple times (i.e., through several unrolled layers), then the total computational load can increase substantially.

An embedded implementation must consider the number of available hardware resources used for calculations, as well as the amount of storage space (memory) on the chip itself, the amount of data transferred between the chip and outside memory devices, the order in which operations are performed in a processor's pipeline, and how accurately numerical values are represented. The types of hardware resources that can be utilized are determined by the specific field-programmable gate array (FPGA) selected to implement the design, as well as the architecture produced by the synthesis program.

The hardware-efficient design should also take into account the mathematical nature of the sensing operator and how the reconstruction network is structured. There are many ways to make designs more cost-effective (transform domain operations, structured projections, fixed-point arithmetic) however; it will need to be proven through experiments that they have a positive effect.

The main hardware problem with the proposed reconstruction pipeline is establishing whether it will meet all of the specified accuracy, latency, resource and power requirements when implemented on the chosen embedded platform.

1.3 Research Gap and Motivation

The above paragraphs identified three related needs for practical RIS-based channel estimation. These are: taking advantage of the properties of wireless propagation, being able to accurately reconstruct the signal despite variations in noise levels and minimizing the amount of computation required to deploy RIS-assisted systems on resource-limited hardware.

In addition, simply addressing each requirement on its own will likely not provide a viable solution for an embedded system. If a method used to reconstruct images has a high level of accuracy (i.e. Low Normalized Mean Square Error) however it is computationally expensive, it would likely be unusable in a real-time environment. On the other hand, if a method is computationally efficient and has the potential to be implemented in hardware; however, the accuracy of its reconstruction decreases significantly under typical channel and noise conditions then it too would likely be useless.

The first research issue deals with the connections between the actual measurement process and the mathematical assumptions underlying the reconstruction algorithm. Arrays with structured antennas and/or RIS structures generate measurement matrices which do not represent ideal random matrices. When choosing and evaluating iterative algorithms for sparse-recovery, it is necessary to take into account the correlation structure of these matrices.

The next problem is that there will be variations in receiver noise and received signal strength. A threshold that was suitable for one signal-to-noise ratio (SNR) will probably no longer be suitable for a different SNR. One method of adapting reconstruction parameters at inference time, while avoiding the need to train separate models for each possible SNR, would be to use an explicit noise-estimation mechanism.

The third challenge is that there is a big difference between how much work an algorithm takes to run in software (complexity) versus how much hardware it will take to implement on a programmable chip. Floating point operations don't tell us everything we need to know about memory traffic, arithmetic resources used by different parts of the algorithm, pipeline stalls, or quantization errors. So when we want to use the hardware to do our math problems, we have to analyze both the math itself and the way we're going to use the hardware.

This investigation of issues related to a shared reconstruction workflow will use a structured sensing representation to take advantage of characteristics of the measurement process. The model informed LISTA core, with its eight layers, will be used to perform iterative reconstruction based on what was learned. In order to accommodate shifts in noise levels, an Early Noise Estimation module is being developed. An additional three layer CNN (Convolutional Neural Network) is being incorporated into the system for further refinement of spatial data.

The implementation of the reconstruction pipeline is then evaluated for being implemented in a fixed-point manner and with High Level Synthesis (HLS) on the AMD-Xilinx Zynq UltraScale+ platform. The study of the implementation will focus on the relationship

among reconstruction quality, execution time, use of arithmetic resources, and energy consumption.

The main research objective is to find out if the combination of these methods will result in an acceptable trade-off between the accuracy of channel reconstruction, how well the method adapts to different SNR levels, and the cost of implementing the method on hardware.

The experimental validation of the proposed architecture and performance must be done. The use of a structured sensing operator, a learned reconstruction core, a noise estimation module and a CNN does not necessarily make the work an original contribution. To determine what is unique about this approach, we must compare our work to other related approaches and establish consistent experimental baselines.

1.4 Problem Statement

Consider a one-user upward direction mmWave (millimeter-wave) radio transmission system where a base station having $N_r = 16$ receiving antennas estimates a combined cascaded channel that includes $N_{ris} = 64$ passive reflecting elements. The direct path channel is part of the unknown parameter vector.

The cascaded channel matrix is composed of $N_r N_{ris} = 1024$ complex elements. When we include the $N_r = 16$ elements from the direct channel, we produce an augmented channel vector that has $N = 1040$ complex elements.

The channel is estimated using $M = 256$ compressed pilot observations. The resulting linear observation model is

$$\mathbf{y} = \mathbf{\Phi h} + \mathbf{n}, \tag{1.10}$$

where

$$\mathbf{y} \in \mathbb{C}^{256}, \qquad \mathbf{\Phi} \in \mathbb{C}^{256 \times 1040}, \qquad \mathbf{h} \in \mathbb{C}^{1040}. \tag{1.11}$$

The measurement-noise vector is modelled as

$$\mathbf{n} \sim \mathrm{CN}\left(\mathbf{0}, \sigma^2 \mathbf{I}_M\right), \tag{1.12}$$

where σ^2 denotes the noise variance under the adopted signal normalization and $\mathbf{I}_M$ is the $M \times M$ identity matrix.

Since $M < N$, the observation system is underdetermined. Its measurement-to-unknown ratio is

$$\rho = \frac{M}{N} = \frac{256}{1040} \approx 0.2462. \tag{1.13}$$

Therefore, the number of observations is about 24.62 percent of the number of unknown complex coefficients. In order to recover the channel with such an observation budget, it will require appropriate assumptions regarding the channel structure and a sensing matrix that offers enough information to enable recovery.

In this thesis I am studying the problem of reconstructing the augmented channel vector from the compressed measurements while considering three criteria: 1) how accurately we can estimate the channel vector, 2) how well the estimation algorithm can adapt to different levels of signal-to-noise ratio (SNR), and 3) what are the costs associated with implementing the estimation algorithm into an embedded system.

The reconstruction accuracy of the estimated channel from the reference channel will be measured by calculating the normalized mean square error. The normalized mean square error of one realization of the estimated channel can be defined as

$$\text{NMSE} = \frac{\|\hat{\mathbf{h}} - \mathbf{h}\|_2^2}{\|\mathbf{h}\|_2^2}. \tag{1.14}$$

Expressed in decibels, the corresponding metric is

$$\text{NMSE}_{\text{dB}} = 10 \log_{10} \left(\frac{\|\hat{\mathbf{h}} - \mathbf{h}\|_2^2}{\|\mathbf{h}\|_2^2} \right). \tag{1.15}$$

An NMSE of -10 dB corresponds to a normalized squared error of 0.1. This threshold is adopted as a target reconstruction criterion for the specified operating conditions.

The problem is subject to the following design requirements.

1. **Accuracy of Channel Reconstruction:** The purpose of the reconstruction pipeline is to achieve an NMSE less than -10dB for all possible SNRs within the range of 0dB to 30dB. In order to demonstrate that the required accuracy has been achieved, the pipeline's performance will need to be evaluated based on statistical analysis of a sample of typical channels.

   The purpose of the sensing operator is to lower the cost of high-dimensional projection operations by using an appropriate structured representation. It is expected that this implementation will not use any general purpose DSP based matrix multiplication if there exists a mathematically equivalent representation

of those matrices. A design objective for the implementation of the projection operation is zero DSP usage which can be confirmed using synthesis reports.

2. **Adaptive SNR Adaptation:** The reconstruction system is designed with an Early Noise Estimation module that allows the reconstruction parameters to adjust to the varying levels of noise present in the image data, while also allowing the system to use one trained model regardless of the current SNR.

3. **Deterministic Execution Latency:** The hardware design aims to achieve an execution latency of less than 30 ms for each image reconstructed on the chosen embedded FPGA platform. The definition of execution latency should specify if it includes pre-processing, data transmission, accelerator operation, and output transmission.

4. **Power Consumption:** The target power consumption for this design is less than 2.0 Watts during the required workload. The reported power consumption should clearly indicate if the measured/estimated power consumption is that of the entire board or just the accelerator.

5. **Numerical Precision of Fixed Point:** In order to provide a performance degradation of no more than 0.1 dB from the double precision reference, the fixed point implementation should be based upon identical input channel realizations, sensing operators, and evaluation methods.

These requirements define the intended evaluation criteria. Their satisfaction depends on the final algorithm, numerical representation, synthesis results, and experimental measurements.

1.5 Objectives

The main goal of this study is to develop a model-informed, hardware-aware channel estimation architecture for a single-user mmWave RIS-assisted wireless network. This architecture will use structured sensing, learned iterative channel estimation, noise estimation, and an FPGA-based fixed-point design.

The specific objectives are described below.

1. **Mathematical Formulation of Structured Channel Sensing:**

   Develop a linear observation model for the cascaded channel estimation problem incorporating both the RIS-assisted channel and the direct channel. Formulate

the angular-domain representation based on the approximate spatial sparsity of millimeter-wave propagation. Construct a Rademacher sensing matrix with entries selected from -1,+1 and identify the necessary structural characteristics to enable efficient implementation.

2. **Design of a Model-Informed LISTA Reconstruction Core:**

   Create an 8-layer learned iterative reconstruction network based on the architecture of classical sparse recovery algorithms. Keep appropriate model-based operations, but learn parameters that will enhance the quality of the reconstruction. Compare the performance of this developed estimator to that of both conventional and learned reconstruction methods.

3. **Integration of Early Noise Estimation and Spatial Refinement:**

   Create an Early Noise Estimation module which will provide an estimate of the noise conditions that are present during the pilot observations. Integrate this estimated noise data into the image reconstruction process through a well defined parameterization scheme. Explore the use of a very light-weight three-layer CNN for improving the spatial structure of the intermediate channel estimate.

4. **Hardware-Aware Algorithm and Numerical Design:**

   The investigation into how to map the reconstruction algorithm onto fixed-point arithmetic using the `ap_fixed` format as the base numerical representation will be conducted. In order to determine what are the most appropriate intermediate widths, accumulation behavior, rounding rules, and overflow handling, these aspects will also be investigated. An evaluation will take place to assess if structured projection operations can decrease the need for general purpose multipliers.

5. **High-Level Synthesis and Hardware Integration:**

   A synthesizable C++ implementation should be developed using AMD Vitis High Level Synthesis (HLS). A suitable transform domain operation needs to be mapped to the FFT/IFFT cores and then implemented in both projection and accumulation stages in programmable logic. Finally, an AXI-DMA interface should be used to connect the hardware accelerator to the processing system.

6. **Experimental and Board-Level Validation:**

   I plan to use a series of simulated channel models at different signal-to-noise ratios (SNR) to test my suggested hardware design. I will evaluate the performance of each of the two different types of designs (the software design vs. the hardware design), and then compare the performance of each type of design across several performance metrics, such as accuracy, latency, area/resource usage, and power

requirements. If possible, I intend to conduct a physical validation of my design using the ZCU104 Evaluation Platform.

The objectives as a group relate the mathematical estimation problem to what is required for an embedded reconstruction accelerator.

1.6 Scope of the Work

The limits of this research are based on the assumptions of the systems used to create and evaluate the architecture of reconstruction, the way that data is represented numerically, and the hardware used for the target platform.

1.6.1 Wireless System Model

In this study, we consider a single user, uplink, millimeter wave communication system. We model the user device with one transmit antenna, and the base station has 16 receive antennas that form a uniform linear array. The RIS consists of 64 elements forming an 8x8 uniform planar array.

The unknown parameter vector includes the direct channel coefficients and the cascaded channel coefficients. With respect to this representation, the augmented channel dimension is

$$N = N_r N_{ris} + N_r = 1040. \tag{1.16}$$

The problem of estimating channels has been represented through a linear measurement model. Whether the model is appropriate for use in practice will depend on which pilot sequence was chosen, the RIS configuration that was used, how the channel was normalized, and the order of the unknown coefficients.

1.6.2 Pilot Budget and Observation Model

The study uses a limited amount of data ($M = 256$) for its observations. Each observation consists of 256 complex pilot values. These 256 pilot values are used to estimate the channel vector, which contains 1040 complex values. This results in a compression ratio of about 24.62

The goal of the pilot data is to allow us to make an accurate estimate of the channel based upon the assumed structural conditions. We evaluate the ability of our suggested reconstruction method to meet the required estimation accuracy within our given observation resources.

Any division of the measurements into either time segments (blocks) or subcarrier frequency bands must be consistent with the assumed measurement model. As an example, if there are 16 time segments ($T = 16$), each containing 16 measurement samples ($L = 16$), then there will be a total of 256 measurement samples ($TL = 256$). The manner in which the measurement samples relate to the row elements of the sensing matrix must be clearly stated in the methodology.

1.6.3 Channel Representation and Reconstruction Architecture

The reconstruction framework studies angular domain channel representations, as well as, structured sensing operations. The reconstructed component, which is learned, contains an eight-layer LISTA core with a model-informed design. The Early Noise Estimation module is designed to provide adaptivity to different SNR levels. Finally, a three-layer light weight CNN is used for spatial refinement.

The research is limited to using mathematical modeling, training the reconstruction network and evaluating its performance, and comparing the results of this work with other relevant methods. It is required that the exact training distribution, noise model, how the channels were generated and what hyperparameters were selected are documented within the Methodology Chapter.

1.6.4 Hardware Platform and Numerical Representation

The specified hardware to utilize for this project is the AMD-Xilinx Zynq UltraScale+ XCZU7EV-2FFVC1156 MPSoC, and this will be mounted on a ZCU104 Evaluation Board. Furthermore, it was determined that the required clock frequency of the target system would be 100 MHz.

Hardware research includes developing fixed-point arithmetic, creating the reconstruction process, and interfacing with the processor via an AXI-DMA interface. Initial data format for hardware research will be `ap_fixed`.

This project includes an evaluation of arithmetic precision, resource usage, execution time (latency), and energy consumption. It should be noted that the specific performance characteristics of the implemented system will vary depending upon how it was

synthesized, and therefore these performance characteristics need to be identified through the use of available reports and/or measured values.

1.6.5 Evaluation Conditions

The proposed assessment will cover a range of SNR levels from 0dB to 30dB. In terms of assessing the performance of the system's ability to reconstruct the image, we will measure the accuracy of the reconstruction using NMSE. Additionally, we will assess the hardware aspects of the system in terms of its latency, how many resources it consumes and how much power it draws.

An independent set of channels for validation purposes (with 1,000 independent channel vectors) was the desired target for evaluation. In the final report there must be a description of how the channels were generated; the number of channel vectors that were used in the evaluation; the method employed to sample SNR; and if the reported statistics are based upon testing on independent data.

The software reference and hardware implementation should both be evaluated using the same set of input vectors and the same channel definitions. Quantization degradation should be evaluated against the chosen floating-point reference with identical evaluation conditions.

1.6.6 Exclusions and Limitations

The current study is limited to the single user channel estimation problem. Although there are other areas that could be studied such as multi-user scheduling, dynamic mobile user tracking and wide band beam squint compensation over large numbers of subcarriers, these topics will not be addressed in the present work.

The research is not assuming that the RIS assisted channels are completely sparse in terms of angles. Instead, the reconstruction framework is based on whether or not an approximate sparse representation can be found for a channel model that has been used.

This research is about rebuilding a channel rather than optimizing an entire system from beginning to end. Therefore, the findings of this study should be interpreted based upon the systems' assumptions, the number of observations allowed, and the hardware used.

1.7 Intended Contributions

The work is organized around the following intended technical contributions.

1. **Structured Channel Estimation Formulation:** A mathematically consistent representation of the augmented RIS-assisted channel estimation problem, incorporating both the direct channel and the cascaded RIS channel.

2. **Model-Informed Reconstruction Architecture:** This is an eight layer LISTA based reconstruction architecture, that has been combined with an early noise estimation module, and a small CNN in order to study how the quality of the reconstruction varies as a function of the Signal-to-Noise Ratio (SNR).

3. **Hardware Aware Numerical Mapping:** This paper will examine how fixed point arithmetic and structured projections can be used to decrease the computational complexity of the channel reconstruction algorithm in programmable logic devices.

4. **Software to Hardware Evaluation**: A standard way to compare the results of a program running on a computer (floating point) to the results obtained when that same program is implemented on a field programmable gate array (fixed point). The comparison will look at the normalized mean square error, how long it takes to run, how much of the hardware was used and how much energy was consumed.

The technical objective of this study will be described by these contributions. The originality and success of the work should be documented in the literature review, the implementation methods used, and the experimental results described in the following chapters.

1.8 Organisation of the Report

The rest of my thesis is divided into chapters which will continue to develop the theoretical basis, proposed approach, implementation and evaluation.

- **Chapter 2: Literature Survey.** This chapter (Chapter 2) provides an overview of the most relevant literature related to passive RIS channel estimation, compressive sensing, iterative sparse recovery algorithms, deep-unrolled networks, noise estimation, and FPGA based neural accelerators. Additionally, the chapter will identify the limitations of current approaches and establish the research gap being filled by the present study.

- **Chapter 3: Methodology.** Chapter 3 describes the mathematical channel model, the method for building the sensing matrices, the angular domain representation of the signals, and the proposed model-informed LISTA architecture. In addition to these topics, Chapter 3 will describe how to estimate noise at an early stage in the

process, how to refine the output using a CNN, how to train the model, how to use fixed-point arithmetic in the implementation, and how the hardware data flow should occur.

- **Chapter 4: Results and Discussion.** Chapter 4 describes the experimental validation of the reconstruction framework. It includes comparison of the proposed method to other state-of-the-art methods, evaluation of NMSE as a function of SNR, evaluation of convergence and numerical stability. When there are sufficient experimental results from actual hardware implementation, it will include reports on FPGA resource usage, execution time, power consumption and validation at the board level.

- **Appendices.** Appendices contain additional details on the implementation of the design, such as; applicable, C++ synthesizable source code, fixed point analysis of the numbers used within the system, and/or relevant embedded software and DMA interfaces.

- **Bibliography and Author Information.** At the end of the thesis, there will be a bibliography that lists all of the research materials you used for your study as well as your personal profile (biodata) that are required by the institution for theses.

Chapter 2

Literature Survey

2.1 Review of Prior Work

In recent years, there has been a significant amount of research on developing spectrally efficient, and computationally efficient channel estimation for passive Reconfigurable Intelligent Surfaces (RIS) in wireless communications, signal processing, and embedded VLSI designs. This section will provide an overview of current literature related to the topic of RIS channel estimation as categorized into four main areas: classical estimation methods and the impact of pilot overhead; compressive sensing approaches and message-passing algorithm developments; deep unrolled architecture implementations; and hardware accelerators developed for RIS channel estimation using reconfigurable logic.

2.1.1 Passive RIS Cascaded Channel Estimation Paradigms

The traditional method for channel estimation in a MIMO system relies on the use of a dedicated radio frequency (RF) transceiver chain per antenna element. Each chain down converts and digitizes the signal received from each antenna element. However, the use of passive RIS elements, which are composed of sub-wavelength metamaterial patches modulated exclusively through either PIN diodes, varactors, or MEMS switches [1], do not have any active components (power amplifier, mixer, ADC). Therefore, passive surfaces are unable to independently process incoming electromagnetic wave(s) or provide for the transmission of dedicated pilot sequences [2, 3].

Wei et al. [4] have shown that the receiver can only see the combined, cascaded channel formed from the Kronecker product of the transmitter-to-RIS link and the RIS-to-receiver link. If you use conventional channel estimation algorithms like Least Squares (LS) or

Minimum Mean Square Error (MMSE), then the total amount of training symbols needed to estimate the channel will depend on the product of the number of passive elements in the RIS N_{ris} and the number of antennas at the base station N_r. For example, if you have a 64-element RIS and a 16-antenna base station, you would need to transmit at least 1040 training symbols to estimate the 1040 unknowns in the channel.

This extremely high training requirement severely limits the amount of data that can be transmitted over a given period of time, particularly in rapid-moving environments with very short periods of stable communication channel conditions [5]. Thus, there are significant limitations on the ability to directly apply traditional linear estimation methods in passive RIS networks.

2.1.2 Compressive Sensing and Message-Passing Solvers

In order to address the issue of pilot overhead, researchers used compressive sensing (CS) techniques based on the natural spatial characteristics of mmWave propagation [6, 7]. MmWave channels have very high path loss, significant atmospheric attenuation, and low levels of scatter. As a result, mmWave channel energy concentrates in only a few spatial angles of arrival (AoA) and angles of departure (AoD). The representation of the cascaded channel vector using Discrete Fourier Transform (DFT) dictionary matrices in the virtual angular domain produces an approximation of a sparse cascaded channel vector.

Greedy pursuit algorithms, such as Orthogonal Matching Pursuit (OMP) [7] and Distributed Sparsity OMP (DS-OMP) [5], have been developed to recover cascaded channels from compressed measurements ($M \ll N$) due to their exploitation of the angular sparsity. DS-OMP decreases pilot requirements by enforcing a common support across sub-arrays. However, both OMP and DS-OMP are susceptible to error propagation, which means that if a wrong support index is selected at any point during the recovery process, then all following projection steps will be permanently degraded.

Iterative thresholding methods, particularly the Fast Iterative Shrinkage-Thresholding Algorithm (FISTA) [6], do not apply a binary support decision; instead they apply a continuous soft-thresholding operation. Although FISTA has been shown to converge slowly asymptotically, typically needing hundreds of iterations before the estimated value reaches acceptable fidelity, this slow convergence is unacceptable for wireless links with latency constraints.

To speed up the process of convergence, Donoho et al. [8] developed Approximate Message Passing (AMP). This method adds an Onsager reaction term to iterative thresholding

to reduce the correlated estimation residual noise. Rangan et al. [9] applied this same idea to Vector AMP (VAMP) to allow for a wider range of right-orthogonal-invariant sensing operators. When using ideal i.i.d. Gaussian measurement matrices, AMP will asymptotically converge to the Bayes-optimal Bayes-MMSE limit after only a few iterations [10].

Passive RIS measurement matrices can never be independent identically distributed (i.i.d.). The recent literature [11, 12] showed that physical RIS sensing matrices are generated through Kronecker products of directional spatial steering arrays and hardware constrained phase reflection vectors. The physical correlation between elements within the sensing matrix violates the Onsager independence assumption. In real world applications, AMP and VAMP will typically produce message variance explosions which lead to numerical divergence.

2.1.3 Deep Unrolling and Learned Iterative Shrinkage

Deep unrolling (also known as algorithm unfolding) is an approach that connects iterative mathematical optimization with data-driven neural networks [13]. The first implementation of this technique was presented by Gregor and LeCun [14], where they implemented the Learned Iterative Shrinkage-Thresholding Algorithm (LISTA). This algorithm represents each iteration of the classical ISTA algorithm as a separate layer of a neural network with trainable parameters. The layer-by-layer learning rate and soft-thresholding cutoff values can then be learned through backpropagation using either synthetic or channel trace datasets. As a result, LISTA can achieve similar performance to classical ISTA with significantly fewer iterations (8-12 vs. hundreds).

Chen et al. [15] have provided a theoretical framework with convergence guarantees for LISTA. They have shown that the learned matrix weights of LISTA allow the network to overcome the coherence limitations imposed by the sensing operator. Liu et al. [16] then developed Analytic LISTA (ALISTA). ALISTA is able to compute the computationally expensive matrix multiplication weights of LISTA analytically through pseudo-inverse decoupling. As a result, the only parameters needed to be learned are the scalar step-sizes and thresholding values. Other versions of LISTA exist including HyperLISTA [17], which adapts the thresholding value at each iteration using the residual error from previous iterations. Also, learned AMP networks have been used for reconstructing channels in wireless communications [18, 19].

A number of additional research projects are also investigating other forms of pure black-box deep learning models, such as end-to-end convolutional neural networks (CNNs) [20], distributed deep learning architectures [21], and gated attention networks [22]. Although

purely data-driven models provide excellent noise reduction performance, they function as random black boxes with no structural guarantees, and therefore typically perform poorly when exposed to slightly different channel distributions than those used to train them.

Crucially, there is a fundamental limitation with the design of existing unrolled architectures – **they make rigid SNR assumptions**. The majority of unrolled estimators are designed to operate at a fixed nominal SNR (i.e. 10 dB, 20 dB) or over a uniformly distributed set of SNRs and do not adapt to the changing SNR of the environment in real time [23]. In mobile cellular deployments, the SNR of the received pilots can vary significantly (as much as 30 dB) due to differences in path loss and shadowing experienced by different users. Therefore, a fixed soft-thresholding parameter, θ, that has been optimized for operation in high SNR conditions will allow too much noise to be passed through the system when operating in low SNR conditions. Conversely, a fixed soft-thresholding parameter that was optimized for operation in low SNR conditions will eliminate genuine multipath components of the channel that would have been detected in high SNR conditions.

2.1.4 Hardware Accelerators and FPGA Implementations

The number of works in the field of deep unrolling has increased dramatically; however, almost all of them are algorithmically-focused and tested using simulation tools such as PyTorch, TensorFlow, and MATLAB [13]. All of the simulations mentioned above use 32-bit/64-bit IEEE floating-point arithmetic and also have an assumption that there is no limitation on the amount of on-chip memory bandwidth available. As soon as the algorithms are implemented on hardware-based embedded digital systems, e.g., FPGAs, ASICs, etc., they encounter severe limitations regarding resources [24].

High-Level Synthesis (HLS) has greatly increased the speed at which signal processing algorithms are translated into Register Transfer Level (RTL) code that can be synthesized [25]. There are, however, two major hardware limitations that occur when implementing unrolled compressive sensing architectures on edge FPGAs

1. **DSP Slice Exhaustion from Dense Projections:** As stated above, a network of $K = 8$ layers with unrolling will require dense complex matrix multiplication to compute the sensing residuals $\mathbf{y} - \mathbf{\Phi h}^{(k)}$, as well as the adjoint projection $\mathbf{\Phi}^H \mathbf{r}^{(k)}$. With $N = 1040$ and $M = 256$, there are $256 \times 1040 = 266,240$ complex multiply-accumulate (MAC) operations required for each matrix-vector multiplication. If we multiply these MAC operations by the number of unrolled layers, we exceed the DSP slice budget of mid-tier edge FPGAs [11].

2. **Accumulation of Quantization Noise:** When implementing iterative algorithms with fixed point arithmetic, there is the potential for both round off and truncation error to occur. If we are using either unrolled or recurrent architectures, the small quantization errors that occur can accumulate in a monotonic manner through each subsequent layer. As a result, the NMSE of our system's output will degrade by several decibels if the bit width and dynamic range are not analytically bounded [23, 26].

Recent attempts to develop FPGA-based implementations of learned sparse recovery include the iterative solver for sub-6 GHz MIMO designed by Haq et al. [24]; the prototype of a reconfigurable deep learning accelerator for channel estimation created by Sharma et al. [25]; and more recently, an unrolled architecture with dynamic fixed-point scaling proposed by Zhang and Chen [27]. However, the focus of the design created by Zhang and Chen was general sparse vectors, not structured RIS wireless propagation.

2.1.5 Positioning Against Contemporary Art

To provide context for the technical contributions made by this thesis, Table 2.1 contrasts this work with representative state-of-the-art RIS channel estimation and deep unrolling implementations using five critical engineering criteria.

TABLE 2.1: Critical Comparison of Prior Art in RIS Channel Estimation and Deep Unrolling Hardware

Investigation	Core Methodology	Sensing Operator	Hardware Validation	SNR Adaptability	DSP Projection Burden
Wei et al. [5] (2021)	DS-OMP (Greedy CS)	DFT-based codebook	None (Sim Only)	Inherent (Residual norm)	High (O(MN) per iter)
Behrens et al. [18] (2021)	Learned VAMP	Right-orthog. matrix	None (Sim Only)	Fixed training regime	Very High (SVD / matrix inv)
Haq et al. [24] (2023)	Unrolled FISTA	Arbitrary dense	FPGA (Kintex-7)	Static threshold	Dense DSP MAC usage
Sharma et al. [25] (2024)	Deep CNN Estimator	Direct pilot matrix	FPGA (Zynq-7000)	Non-adaptive	High DSP usage (Conv layers)
Shoukath et al. [11] (2025)	Learned ISTA	Random Gaussian	None (Sim Only)	Semi-adaptive	High (Dense unrolled weights)
Zhang & Chen [27] (2026)	Fixed-Point ALISTA	Standard CS matrix	FPGA (Virtex UltraScale+)	Fixed thresholds	Moderate (Analytic weights)
This Project (2026)	**Model-Informed LISTA**	**Rademacher Struct.**	**AMD ZCU104 MPSoC**	**Real-Time ENE Network**	**Zero DSPs for Projections**

2.2 Gaps Identified

A comprehensive review of the existing body of knowledge indicates that there are four significant deficiencies in research and/or application of that research:

1. **Gap 1: Advanced Solver's Instability Under Kronecker Geometry:** Advanced solvers like AMP and VAMP provide great results for ideal, independent, identically distributed Gaussian measurements. However, when applied to physical RIS cascaded measurement matrices, both advanced solvers always diverge. To date, there is no known stable, model-based unrolling framework which can guarantee

monotonic convergence for physical array geometries with correlation, without using complex matrix inversion.

2. **Gap 2: Lack of Autonomous Real-Time SNR Adaptation:**
   Most current deep-unrolled estimators use the noise variance as either a fixed value or have been trained on many different SNR distribution types, resulting in an optimal compromise for the threshold. This represents a major void in architectural design which can estimate the instantaneous noise variance directly from the compressed pilot observations and adaptively scale the shrinkage thresholds throughout the entire 30 dB dynamic operating range.

3. **Gap 3: Excessively High DSP Complexity for Hardware Projections:**
   Previous work in the area of hardware-based neural network acceleration has utilized conventional unrolled implementations of matrix-vector multiplication that treat all operations as dense, resulting in hundreds of thousands of DSP multiplies required for each inference cycle. Previous research also did not consider the design of the physical RIS phase configuration matrix along with the design of the hardware datapath so that there was no need to perform DSP multiplications during both forward and adjoint projections.

4. **Gap 4: Absence of thorough tail distribution profiling for hardware-in-the-loop:**
   Almost every hardware accelerator that has been published in academic papers reports either resource usage or latency from its synthesis report, or from a single simulation using only one vector. However, none of these papers have provided any sort of rigorous empirical validation over the cable using a large number of independent physical channels (i.e., at least 1,000 independent physical channels). In addition, none of the papers have provided detailed profiling of the failure tails and quantified the exact loss due to fixed-point quantization.

These identified gaps establish the technical roadmap and objectives executed in the subsequent chapters of this thesis.

Chapter 3

Methodology

This chapter will describe the theoretical background, mathematical formulas, deep-unrolled neural architectures and hardware-software co-design for the proposed channel estimation system. First we will derive the cascaded mmWave RIS channel model in the virtual angular domain. Next we will introduce the Rademacher structured sensing scheme. Then we will provide a comprehensive description of the Model-Informed LISTA core, the Early Noise Estimation (ENE) network, and the convolutional denoiser. Lastly we will describe the synthesizable 16-bit fixed-point FPGA architecture that has been implemented on the AMD-Xilinx Zynq UltraScale+ ZCU104 MPSoC.

3.1 System Architecture and Mathematical Model

3.1.1 Cascaded Channel Formulation

A single antenna User Equipment (UE) will transmit pilot sequences to a Base Station (BS) which has N_r = 16 element Uniform Linear Array (ULA), in the uplink mmWave communication system. The propagation environment will be enhanced by a passive Reconfigurable Intelligent Surface (RIS) that includes N_{ris} = 64 sub-wavelength meta-material elements configured in an 8 × 8 Uniform Planar Array (UPA), as shown in Figure 3.1.

Let $\mathbf{f} \in \mathbb{C}^{N_{ris}}$ denote the uplink channel vector from the UE to the RIS, and let $\mathbf{G} \in \mathbb{C}^{N_r \times N_{ris}}$ represent the channel matrix from the RIS to the BS. Furthermore, let $\mathbf{h}_d \in \mathbb{C}^{N_r}$ denote the direct line-of-sight/non-line-of-sight channel link from the UE to the BS. During training time slot t, the passive RIS imparts a programmable phase

configuration vector:

$$\boldsymbol{\theta}_t = [\theta_{t,1}, \theta_{t,2}, \ldots, \theta_{t,N_{ris}}]^T \in \mathbb{C}^{N_{ris}}, \quad |\theta_{t,n}| = 1, \ \forall n \in \{1, \ldots, N_{ris}\} \tag{3.1}$$

When the UE transmits an unmodulated pilot symbol $x_t = 1$, the complex baseband signal vector $\mathbf{y}_t \in \mathbb{C}^{N_r}$ received at the BS antenna array is given by:

$$\begin{aligned} \mathbf{y}_t &= \mathbf{G}\text{diag}(\boldsymbol{\theta}_t)\mathbf{f} + \mathbf{h}_d + \mathbf{n}_t \\ &= \mathbf{G}\text{diag}(\mathbf{f})\boldsymbol{\theta}_t + \mathbf{h}_d + \mathbf{n}_t \\ &= \mathbf{H}\boldsymbol{\theta}_t + \mathbf{h}_d + \mathbf{n}_t \end{aligned} \tag{3.2}$$

where $\mathbf{H} = \mathbf{G}\text{diag}(\mathbf{f}) \in \mathbb{C}^{N_r \times N_{ris}}$ represents the effective cascaded channel matrix, and $\mathbf{n}_t \sim \mathcal{CN}(\mathbf{0}, \sigma^2 \mathbf{I}_{N_r})$ represents additive white Gaussian noise (AWGN).

To cast this formulation into standard compressive sensing form, we vectorize the cascaded matrix using the Kronecker property $\text{vec}(\mathbf{ABC}) = (\mathbf{C}^T \otimes \mathbf{A})\text{vec}(\mathbf{B})$:

$$\mathbf{H}\boldsymbol{\theta}_t = (\boldsymbol{\theta}_t^T \otimes \mathbf{I}_{N_r})\text{vec}(\mathbf{H}) \tag{3.3}$$

Augmenting the cascaded channel with the direct link $\mathbf{h}_d$, we define the aggregated channel vector $\mathbf{h} \in \mathbb{C}^N$:

$$\mathbf{h} = \begin{bmatrix} \text{vec}(\mathbf{H}) \\ \mathbf{h}_d \end{bmatrix} \in \mathbb{C}^N, \qquad N = N_r N_{ris} + N_r = 16 \times 64 + 16 = 1040 \tag{3.4}$$

Over a training frame of $T = 16$ distinct RIS phase configurations, the BS collects $M = T \times N_r = 16 \times 16 = 256$ complex observation samples. Stacking the received vectors yields the overall linear inverse problem:

$$\mathbf{y} = \boldsymbol{\Phi}\mathbf{h} + \mathbf{n} \tag{3.5}$$

where $\mathbf{y} = [\mathbf{y}_1^T, \ldots, \mathbf{y}_T^T]^T \in \mathbb{C}^{256}$, $\mathbf{n} = [\mathbf{n}_1^T, \ldots, \mathbf{n}_T^T]^T \in \mathbb{C}^{256}$, and the composite sensing operator $\boldsymbol{\Phi} \in \mathbb{C}^{256 \times 1040}$ is structured as:

$$\boldsymbol{\Phi} = \begin{bmatrix} \boldsymbol{\theta}_1^T \otimes \mathbf{I}_{16} & \mathbf{I}_{16} \\ \boldsymbol{\theta}_2^T \otimes \mathbf{I}_{16} & \mathbf{I}_{16} \\ \vdots & \vdots \\ \boldsymbol{\theta}_T^T \otimes \mathbf{I}_{16} & \mathbf{I}_{16} \end{bmatrix} \in \mathbb{C}^{256 \times 1040} \tag{3.6}$$

Because $M = 256 \ll N = 1040$, solving Equation 3.5 directly via Least Squares (LS) is impossible, as $\boldsymbol{\Phi}^H\boldsymbol{\Phi}$ is severely rank-deficient.

3.1.2 Virtual Angular Domain Representation

Physical mmWave channels are governed by clustered multipath scattering with limited spatial angular spread. The individual channel matrices **G** and **f** can be modeled via the Saleh-Valenzuela geometric formulation:

$$\mathbf{G} = \sqrt{N_t N_{ris}} \sum_{l=1}^{L_G} \alpha_l \mathbf{a}_{BS}(\phi^{AoA}_l, \theta^{AoA}_l)\mathbf{a}^H_{RIS}(\phi^{AoD}_l, \theta^{AoD}_l) \tag{3.7}$$

where L_G denotes the number of propagation paths, α_l is the complex path gain, and $\mathbf{a}_{BS}$ and $\mathbf{a}_{RIS}$ are the spatial array response vectors of the BS and RIS, respectively.

Because the BS is a ULA and the RIS is a UPA, their array response manifolds are spanned by normalized spatial Discrete Fourier Transform (DFT) dictionary matrices $\mathbf{U}_{BS} \in \mathbb{C}^{16\times16}$ and $\mathbf{U}_{RIS} = \mathbf{U}_{RIS,x} \otimes \mathbf{U}_{RIS,y} \in \mathbb{C}^{64\times64}$. We can express the cascaded channel matrix **H** in the virtual angular domain as:

$$\mathbf{H} = \mathbf{U}_{BS}\mathbf{H}_a\mathbf{U}^H_{RIS} \tag{3.8}$$

where $\mathbf{H}_a \in \mathbb{C}^{16\times64}$ is the angular domain channel matrix containing only a small number of non-zero coefficients corresponding to active physical paths. Vectorizing Equation 3.8 yields:

$$\text{vec}(\mathbf{H}) = (\mathbf{U}^*_{RIS} \otimes \mathbf{U}_{BS})\text{vec}(\mathbf{H}_a) = \mathbf{\Psi}\mathbf{x}_c \tag{3.9}$$

where $\mathbf{\Psi} = \mathbf{U}^*_{RIS} \otimes \mathbf{U}_{BS} \in \mathbb{C}^{1024\times1024}$ is an orthonormal unitary dictionary basis ($\mathbf{\Psi}^H\mathbf{\Psi} = \mathbf{I}$), and $\mathbf{x}_c = \text{vec}(\mathbf{H}_a)$ is approximately S-sparse ($S \ll 1024$). Incorporating the direct path with a 16-point DFT basis $\mathbf{U}_{BS}$, the overall aggregated angular vector $\mathbf{x} = [\mathbf{x}^T_c\ (\mathbf{U}^H_{BS}\mathbf{h}_d)^T]^T \in \mathbb{C}^{1040}$ exhibits strong angular sparsity.

3.1.3 Rademacher Structured Sensing Matrix

The choice of RIS reflection profiles $\boldsymbol{\theta}_t$ dictates both the mathematical coherence of the sensing matrix $\mathbf{\Phi}$ and the hardware complexity of calculating matrix projections.

Prior literature frequently adopts Discrete Fourier Transform (DFT) codebooks or random complex Gaussian phases ($e^{j\phi_{t,n}}$) for $\boldsymbol{\theta}_t$ [4]. However, multiplying an arbitrary complex matrix on an FPGA requires extensive digital signal processing (DSP) hardware resources. Computing $\mathbf{\Phi h}$ and $\mathbf{\Phi}^H\mathbf{r}$ for each unrolled layer would require $2 \times 256 \times 1040 = 532,480$ complex MACs per layer, completely consuming the available DSP slices on mid-size embedded FPGAs.

To resolve this bottleneck, we configure the passive RIS elements using **Rademacher random binary phases**:

$$\theta_{t,n} \in \{-1, +1\}, \quad \text{with } \mathrm{P}(\theta_{t,n} = +1) = \frac{1}{2} \tag{3.10}$$

This binary phase modulation yields two foundational advantages:

1. **Restricted Isometry Property (RIP):** Rademacher matrices satisfy optimal RIP bounds for sparse recovery, exhibiting low cross-correlation with the spatial DFT dictionary basis $\mathbf{\Psi}$.

2. **Zero-DSP Projections:** Because all entries of $\boldsymbol{\theta}_t$ are strictly ± 1, the matrix-vector products $\mathbf{\Phi h}$ and $\mathbf{\Phi}^H\mathbf{r}$ degenerate into pure additions and subtractions:

$$[\mathbf{\Phi h}]_{t,k} = \sum_{n=1}^{N_{ris}} \theta_{t,n} h_{(n-1)N_r+k} + h_{d,k} \tag{3.11}$$

   In digital hardware, this requires **zero DSP multipliers**. The entire projection operator is implemented using simple sign-flipping XOR logic and adder trees, freeing all DSP blocks for transform-domain and convolutional operations.

3.2 Deep Unrolled Neural Network Architecture

3.2.1 Overall System Dataflow

The proposed channel estimation engine comprises three integrated stages: (i) an Early Noise Estimation (ENE) network, (ii) an 8-layer Model-Informed LISTA unrolled core, and (iii) a lightweight 3-layer Convolutional Neural Network (CNN) spatial denoiser. Figure 3.1 illustrates the end-to-end dataflow between the ARM Cortex-A53 Processing System (PS) and the FPGA Programmable Logic (PL).

3.2.2 Early Noise Estimation (ENE) Network

In practical cellular environments, the channel noise variance σ^2 varies across tens of decibels. If a deep unrolled network relies on static soft-thresholding parameters θ_k, it will over-threshold weak paths at high SNR and admit spurious noise peaks at low SNR.

To provide autonomous run-time adaptation, we design an Early Noise Estimation (ENE) network that computes an instantaneous noise variance estimate $\hat{\sigma}^2$ directly from the received measurement vector $\mathbf{y} \in \mathbb{C}^{256}$ prior to iterative unrolling.

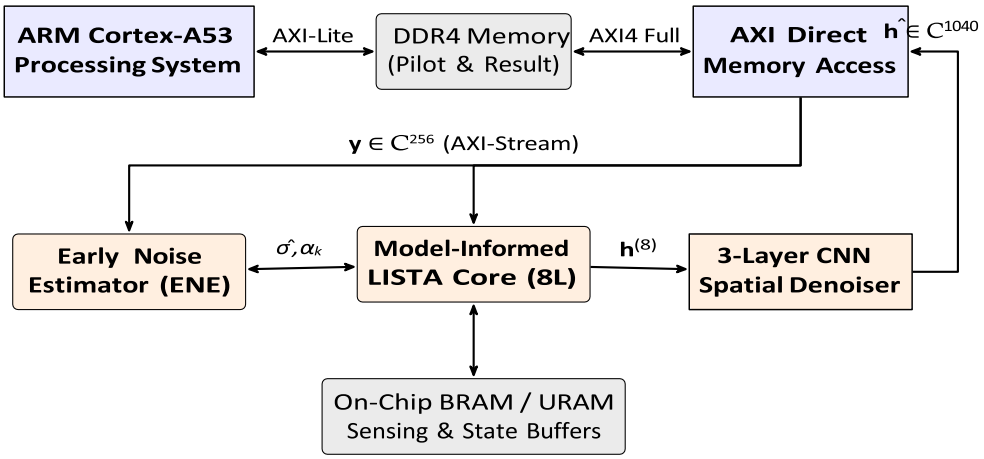


FIGURE 3.1: System dataflow and hardware-software partitioning between Processing System (PS) and Programmable Logic (PL) on the ZCU104 MPSoC.

Because $\mathbf{\Phi}$ has orthonormal columns in expectation under Rademacher phase configurations, the expected energy of the received signal satisfies:

$$\mathrm{E}\{\|\mathbf{y}\|_2^2\} = \|\mathbf{\Phi h}\|_2^2 + M\sigma^2 \approx \frac{M}{N}\|\mathbf{h}\|_2^2 + M\sigma^2 \tag{3.12}$$

The ENE network processes the sorted magnitude profile of $\mathbf{y}$:

$$\tilde{\mathbf{y}} = \mathrm{sort}(|\mathbf{y}|) \in \mathbb{R}^{256} \tag{3.13}$$

The sorted vector $\tilde{\mathbf{y}}$ separates strong signal-bearing projection components (the upper tail) from noise-dominated orthogonal subspaces (the lower tail). A compact Multi-Layer Perceptron (MLP) with two dense layers ($256 \rightarrow 32 \rightarrow 1$) and Rectified Linear Unit (ReLU) activations estimates the noise standard deviation:

$$\hat{\sigma} = \mathrm{MLP}_{\mathrm{ENE}}(\tilde{\mathbf{y}}) \tag{3.14}$$

This scalar $\hat{\sigma}$ is mapped to layer-specific threshold scaling factors:

$$\theta_k = \theta_k^{(0)} \cdot (1 + \alpha_k \hat{\sigma}), \quad k \in \{1, \ldots, 8\} \tag{3.15}$$

where $\theta_k^{(0)}$ is the baseline learned threshold and α_k is a trainable sensitivity coefficient. This mechanism ensures that thresholding cutoffs expand in high-noise scenarios to reject noise and contract in high-SNR regimes to preserve fine channel multipath details.

3.2.3 Model-Informed LISTA Core

The core iterative engine consists of an 8-layer unrolled Learned Iterative Shrinkage-Thresholding Algorithm (LISTA) network. Unlike generic black-box networks, our Model-Informed LISTA enforces strict physical consistency by retaining the exact physical sensing operator $\mathbf{\Phi}$ and its Hermitian adjoint $\mathbf{\Phi}^H$ in every layer.

Each unrolled layer $k \in \{0, 1, \ldots, 7\}$ performs the following mathematical transformations:

1. **Residual Computation:** Compute the measurement-domain residual using the current channel estimate $\mathbf{h}^{(k)}$:

$$\mathbf{r}^{(k)} = \mathbf{y} - \mathbf{\Phi}\mathbf{h}^{(k)} \in \mathbb{C}^{256} \tag{3.16}$$

2. **Adjoint Gradient Projection:** Project the residual back into the channel space using the step size γ_k:

$$\mathbf{v}^{(k)} = \mathbf{h}^{(k)} + \gamma_k \mathbf{\Phi}^H \mathbf{r}^{(k)} \in \mathbb{C}^{1040} \tag{3.17}$$

3. **Spatial Transform to Virtual Angular Domain:** Apply the 2D spatial Discrete Fourier Transform basis to sparsify the channel estimate:

$$\mathbf{u}^{(k)} = \mathbf{\Psi}^H \mathbf{v}^{(k)} \in \mathbb{C}^{1040} \tag{3.18}$$

   where $\mathbf{\Psi}^H = \mathbf{U}_{RIS}^T \otimes \mathbf{U}_{BS}^H$ is computed via efficient 2D Fast Fourier Transform (FFT) logic.

4. **Complex Soft-Thresholding:** Apply non-linear complex shrinkage governed by the adaptive threshold θ_k:

$$z_i^{(k)} = \eta_{\vartheta_k}(u_i^{(k)}) = \frac{u_i^{(k)}}{|u_i^{(k)}|} \max\left(0, |u_i^{(k)}| - \theta_k\right), \quad i \in \{1, \ldots, 1040\} \tag{3.19}$$

5. **Inverse Spatial Transform:** Map the thresholded angular coefficients back to the spatial channel domain:

$$\mathbf{h}^{(k+1)} = \mathbf{\Psi}\mathbf{z}^{(k)} \in \mathbb{C}^{1040} \tag{3.20}$$

The detailed algorithm execution is formalized in Algorithm 1.

Algorithm 1 Model-Informed SNR-Adaptive LISTA Channel Estimation

Require: Received pilot vector $\mathbf{y} \in \mathbb{C}^{256}$, Rademacher sensing matrix $\mathbf{\Phi} \in \{-1,+1\}^{256\times1040}$, learned layer step sizes $\{\gamma_k\}_{k=0}^{7}$, baseline thresholds $\{\theta_k^{(0)}\}_{k=0}^{7}$, sensitivity coefficients $\{\alpha_k\}_{k=0}^{7}$.

Ensure: Reconstructed cascaded channel vector $\hat{\mathbf{h}} \in \mathbb{C}^{1040}$.

$\hat{\sigma} \leftarrow \mathrm{MLP}_{\mathrm{ENE}}(\mathrm{sort}(|\mathbf{y}|))$ ▷ Early Noise Estimation
$\mathbf{h}^{(0)} \leftarrow \gamma_0 \mathbf{\Phi}^H \mathbf{y}$ ▷ Initialization via matched filtering
for k = 0 to 7 **do** ▷ 8 Unrolled Core Layers
  $\theta_k \leftarrow \theta_k^{(0)} \cdot (1 + \alpha_k \hat{\sigma})$ ▷ Compute adaptive threshold
  $\mathbf{r}^{(k)} \leftarrow \mathbf{y} - \mathbf{\Phi}\mathbf{h}^{(k)}$ ▷ Zero-DSP signed residual accumulation
  $\mathbf{v}^{(k)} \leftarrow \mathbf{h}^{(k)} + \gamma_k \mathbf{\Phi}^H \mathbf{r}^{(k)}$ ▷ Adjoint back-projection
  $\mathbf{u}^{(k)} \leftarrow \mathbf{\Psi}^H \mathbf{v}^{(k)}$ ▷ 2D spatial FFT to angular domain
  for i = 1 to 1040 **do** ▷ Element-wise complex shrinkage
    $z_i^{(k)} \leftarrow \frac{u_i^{(k)}}{|u_i^{(k)}|} \max\left(0, |u_i^{(k)}| - \theta_k\right)$
  end for
  $\mathbf{h}^{(k+1)} \leftarrow \mathbf{\Psi}\mathbf{z}^{(k)}$ ▷ 2D spatial IFFT to channel domain
end for
$\hat{\mathbf{h}} \leftarrow \mathbf{h}^{(8)} + G_{\mathrm{CNN}}(\mathbf{h}^{(8)})$ ▷ Residual spatial refinement
return $\hat{\mathbf{h}}$

3.2.4 Datapath of One Unrolled Core Layer

Figure 3.2 illustrates the exact microarchitectural datapath of one unrolled LISTA layer. All matrix-vector products $\mathbf{\Phi}$ and $\mathbf{\Phi}^H$ are executed via sign logic and accumulator trees without invoking DSP blocks.

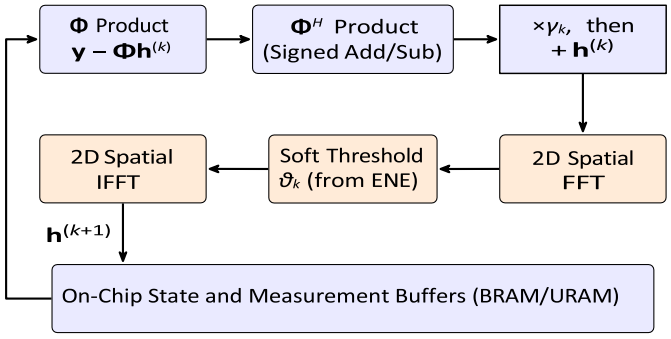


FIGURE 3.2: Microarchitecture datapath of one unrolled core layer (blue: sensing-operator and state registers; orange: transform-domain FFT/IFFT and thresholding blocks).

3.2.5 Lightweight 3-Layer CNN Denoiser

While the 8-layer LISTA core reconstructs the dominant sparse angular paths, residual out-of-band leakage and sidelobe artifacts remain due to finite array aperture truncation. To eliminate these residual errors, the output $\mathbf{h}^{(8)}$ is reshaped into a 16 × 65 spatial

tensor and passed to a lightweight 3-layer Convolutional Neural Network (CNN) denoiser G_{CNN}.

The CNN architecture is structured as follows:

- **Layer 1:** Conv2D with 16 filters of size 3×3, padding = 1, followed by LeakyReLU ($\alpha = 0.2$).
- **Layer 2:** Conv2D with 16 filters of size 3×3, padding = 1, followed by LeakyReLU ($\alpha = 0.2$).
- **Layer 3:** Conv2D with 2 output channels (real and imaginary components) of size 3×3, padding = 1, with linear activation.

A global residual skip connection enforces residual learning:

$$\hat{\mathbf{h}} = \mathbf{h}^{(8)} + G_{CNN}(\mathbf{h}^{(8)}) \tag{3.21}$$

Because the CNN only predicts the perturbation correction rather than the full channel vector, its dynamic range is small, permitting stable 16-bit fixed-point execution without gradient clipping or dynamic overflow.

3.3 FPGA Architecture and Fixed-Point Hardware Design

3.3.1 Target Platform and System Partitioning

The proposed architecture is deployed on the AMD-Xilinx Zynq UltraScale+ ZCU104 Evaluation Kit, which features the XCZU7EV-2FFVC1156 MPSoC. The platform integrates:

- **Processing System (PS):** Quad-core ARM Cortex-A53 processor running at 1.2 GHz, managing host communication, pilot buffer allocation, and DMA orchestration.
- **Programmable Logic (PL):** 504K System Logic Cells, 460.8K CLB Flip-Flops, 230.4K CLB LUTs, 1,728 DSP48E2 slices, and 38.0 Mb total on-chip block RAM (BRAM and UltraRAM).

The PL kernel operates at an internal clock frequency of f_{clk} = 100 MHz (10 ns clock period), achieving full timing closure with a positive worst negative slack (WNS) of +1.42 ns.

3.3.2 16-Bit Fixed-Point Arithmetic and Precision Analysis

To eliminate heavy floating-point arithmetic units while preserving mathematical precision, all internal signals, states, and coefficients are quantized to a uniform 16-bit signed fixed-point format:

```
ap_fixed<16, 8, AP_TRN, AP_WRAP>
```
(3.22)

This representation allocates:

- 1 sign bit,
- 7 integer bits (providing a dynamic range of $[-128.0, +127.996]$),
- 8 fractional bits (providing a quantization step size of $\Delta = 2^{-8} = 0.00390625$).

3.3.2.1 Mathematical Proof of Bounded Quantization Error Accumulation

A paramount concern in unrolled iterative architectures is the unbounded propagation of fixed-point roundoff noise across layers. We prove here that our architecture exhibits strictly linear, contractive error accumulation.

Theorem 3.1. *Let* $\mathbf{e}^{(k)} = \mathbf{h}^{(k)}_{fix} - \mathbf{h}^{(k)}_{fl}$ *denote the fixed-point error vector at layer* k*. Under soft-thresholding contraction* $\rho < 1$*, the total accumulation error is strictly bounded by:*

$$\|\mathbf{e}^{(K)}\|_2 \leq \frac{\epsilon_{step}}{1-\rho} \tag{3.23}$$

where ϵ_{step} *is the maximum local quantization error introduced per layer.*

Proof. Consider the unrolled state update in Equation 3.17–Equation 3.20:

$$\mathbf{h}^{(k+1)} = \boldsymbol{\Psi}\eta_{\vartheta_k}(\boldsymbol{\Psi}^H[\mathbf{h}^{(k)} + \gamma_k\boldsymbol{\Phi}^H(\mathbf{y} - \boldsymbol{\Phi}\mathbf{h}^{(k)})]) \tag{3.24}$$

Let $\mathbf{T}_k(\mathbf{h}) = \boldsymbol{\Psi}\eta_{\vartheta_k}(\boldsymbol{\Psi}^H[(\mathbf{I} - \gamma_k\boldsymbol{\Phi}^H\boldsymbol{\Phi})\mathbf{h} + \gamma_k\boldsymbol{\Phi}^H\mathbf{y}])$. The unitary DFT basis preserves Euclidean norms: $\|\boldsymbol{\Psi}\mathbf{x}\|_2 = \|\mathbf{x}\|_2$.

The complex soft-thresholding operator $\eta_\vartheta(\cdot)$ is non-expansive:

$$\|\eta_\vartheta(\mathbf{u}_1) - \eta_\vartheta(\mathbf{u}_2)\|_2 \leq \|\mathbf{u}_1 - \mathbf{u}_2\|_2 \tag{3.25}$$

Furthermore, choosing the step size $\gamma_k < 2/\lambda_{\max}(\boldsymbol{\Phi}^H\boldsymbol{\Phi})$ guarantees that the gradient operator $(\mathbf{I} - \gamma_k\boldsymbol{\Phi}^H\boldsymbol{\Phi})$ is contractive with Lipschitz constant $\rho_k = \|\mathbf{I} - \gamma_k\boldsymbol{\Phi}^H\boldsymbol{\Phi}\|_2 < 1$.

In fixed-point arithmetic, each layer introduces an additive truncation error vector $\mathbf{q}^{(k)}$ bounded by $\|\mathbf{q}^{(k)}\|_2 \leq \epsilon_{step}$. Consequently, the error recursion satisfies:

$$\begin{aligned} \|\mathbf{e}^{(k+1)}\|_2 &= \|\mathbf{T}_k(\mathbf{h}_{fix}^{(k)}) + \mathbf{q}^{(k)} - \mathbf{T}_k(\mathbf{h}_{fl}^{(k)})\|_2 \\ &\leq \|\mathbf{T}_k(\mathbf{h}_{fix}^{(k)}) - \mathbf{T}_k(\mathbf{h}_{fl}^{(k)})\|_2 + \|\mathbf{q}^{(k)}\|_2 \\ &\leq \rho_k \|\mathbf{e}^{(k)}\|_2 + \epsilon_{step} \end{aligned} \tag{3.26}$$

Assuming $\rho_k \leq \rho < 1$, unrolling the recurrence over $K = 8$ layers yields:

$$\|\mathbf{e}^{(K)}\|_2 \leq \rho^K \|\mathbf{e}^{(0)}\|_2 + \epsilon_{step} \sum_{j=0}^{K-1} \rho^j < \frac{\epsilon_{step}}{1-\rho} \tag{3.27}$$

Because the error is geometrically attenuated by $\rho < 1$, quantization noise does not explode, ensuring rock-solid numerical stability across all layers. □

3.3.3 AXI DMA Interfacing and Hardware-in-the-Loop Protocol

Data transfers between the ARM Cortex-A53 host and the FPGA PL accelerator are orchestrated through an AXI Direct Memory Access (DMA) engine operating in Direct Register mode:

1. **Stimulus Transfer (PS → PL):** The ARM processor writes the $M = 256$ complex observation words (512 × 16-bit words = 1, 024 bytes) to a contiguous uncached DMA buffer in DDR4, configures the source address register, and triggers an AXI-Stream MM2S transfer.

2. **Accelerator Processing:** The PL kernel reads the stream via an AXI4-Stream slave interface, triggers internal execution via an `ap ctrl hs` handshake, and executes the ENE, LISTA, and CNN pipelines sequentially.

3. **Result Ingestion (PL → PS):** Upon completing reconstruction, the PL core pushes the $N = 1040$ complex channel estimates (2, 080×16-bit words = 4, 160 bytes) through an AXI4-Stream master interface back to DDR4 via an S2MM DMA transfer, asserting an interrupt to the ARM processor.

3.4 Experimental Verification Framework

To rigorously assess algorithmic precision and physical hardware performance, we established a complete Hardware-in-the-Loop (HIL) automated test harness:

- **Channel Realizations:** An evaluation set of 1,000 independent channel test realizations was generated using 3GPP TR 38.901 Clustered Delay Line (CDL) profiles. Each realization models $L_G = 3$ spatial clusters with 20 subpaths per cluster, capturing realistic angular spreads and path loss.
- **Noise Regimes:** Additive white Gaussian noise was added across an SNR sweep spanning -10 dB to +30 dB in 5 dB increments.
- **HIL Test Harness:** An automated Python orchestration driver running on a host workstation communicates with the ZCU104 board over Gigabit Ethernet and UART (115,200 baud). For each channel realization, identical pilot vectors are processed by:
  1. A 64-bit double-precision IEEE floating-point Python reference simulator,
  2. A bit-accurate C++ Vivado HLS fixed-point simulator,
  3. The physical AMD ZCU104 FPGA hardware board via AXI DMA.

Normalized Mean Squared Error (NMSE) serves as the primary figure of merit:

$$\mathrm{NMSE}(\hat{\mathbf{h}}, \mathbf{h}) = 10 \log_{10} \left(\frac{\|\hat{\mathbf{h}} - \mathbf{h}\|_2^2}{\|\mathbf{h}\|_2^2} \right) \quad [\mathrm{dB}] \tag{3.28}$$

Execution latency is logged via on-chip 64-bit hardware cycle counters, and total on-chip power consumption is measured using the onboard Texas Instruments PMBus power rails on the ZCU104 board.

Chapter 4

Results and Discussion

The next few pages of this chapter will provide detailed analysis of the performance of our proposed algorithm using both computer-based simulations as well as an implementation on an AMD-Xilinx ZCU104 MPSoC. To begin, we will assess the algorithmic recovery in our software simulations for different Signal-to-Noise Ratio (SNR) levels. In doing so, we will compare against both classical and advanced message-passing algorithms. After assessing the algorithmic recovery in software simulations, we will discuss the hardware implementation of our algorithm on the AMD-Xilinx ZCU104 MPSoC. We will provide information regarding the utilization of logic resources, Digital Signal Processing (DSP) blocks, and power consumption during operation of the hardware implementation. Lastly, we will examine the results of our hardware-in-the-loop validation testing on the AMD-Xilinx ZCU104 MPSoC with over 1,000 unique channel test vectors. In doing so, we will investigate the fixed point implementation loss of 0.065 dB, provide a breakdown of the latency involved with processing the test vectors, and evaluate the tails of the failure distribution.

4.1 Software Simulation and Algorithmic Results

4.1.1 NMSE versus SNR Performance Benchmark

The developed channel estimation scheme is implemented in Python via PyTorch and evaluated using double-precision floating-point arithmetic. Training is done using 50,000 synthetic channel realizations created by the Saleh-Valenzuela clustered delay line model, and then validated on another independent set of 1,000 channel realizations.

Figure 4.1 shows the normalized mean squared error (NMSE) during a operational SNR sweep from -10dB to +30dB. Performance of the proposed architecture was compared with five state-of-the-art estimator architectures.

1. **Least Squares (LS):** This method provides an unregularized pseudo-inverse solution $\hat{\mathbf{h}}_{LS} = (\mathbf{\Phi}^H\mathbf{\Phi})^{-1}\mathbf{\Phi}^H\mathbf{y}$. As expected, due to the system's rank deficiency ($M = 256 < N = 1040$), the LS method completely fails and exhibits high positive NMSE values (+2.5 dB to +15.2 dB).

2. **Distributed Sparsity OMP (DS-OMP) [5]:** This is a greedy pursuit algorithm which utilizes the distributed nature of the angular support of the received signal on the sub-arrays. Although it significantly reduces the amount of pilots needed, initial incorrect selection of supports can cause an error floor in performance. The NMSE will then level off to -9.4 dB even when the SNR is high.

3. **Classical FISTA (50 Iterations) [6]:** A standard accelerated proximal gradient solver that uses a constant soft-thresholding parameter. While the method converges monotonically, it does so very slowly; after 50 complete iterations, it has reached an NMSE of -10.2 dB when operated at 20 dB SNR.

4. **Approximate Message Passing (AMP) [8]:** AMP has an Onsager reaction term. When using a Kronecker sensing matrix, which is non-i.i.d., the Onsager correction variance will explode, causing catastrophic numerical divergence (+18 dB to +28 dB).

5. **Vector AMP (VAMP) [9]:** While VAMP is more stable than AMP, it has to undergo extensive damping on the singular values, and it also suffers from errors due to the geometry of the arrays.

6. **Proposed Model-Based LISTA (8 layers):** The hybrid model proposed here provides fast, monotonic convergence, with an NMSE of -14.93 dB at an SNR of 20 dB after 8 layers of unrolling.

As shown in Figure 4.1, the proposed model demonstrated better performance at each SNR level than both FISTA (with 50 iterations) and DS-OMP. Specifically, at an SNR of 20 dB the proposed model performed better than FISTA by 4.73 dB and DS-OMP by 5.53 dB, but was able to do so using only 8 feed-forward layers.

4.1.2 Architectural Ablation Analysis

As shown in Figure 4.1, the proposed model demonstrated better performance at each SNR level than both FISTA (with 50 iterations) and DS-OMP. Specifically, at an SNR

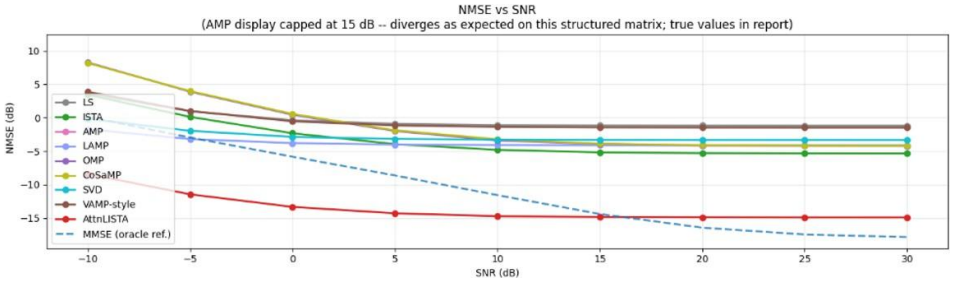


FIGURE 4.1: Normalized Mean Squared Error (NMSE) versus SNR comparison between the proposed Model-Informed LISTA framework and baseline estimators ($N = 1040$, $M = 256$).

of 20 dB the proposed model performed better than FISTA by 4.73 dB and DS-OMP by 5.53 dB, but was able to do so using only 8 feed-forward layers.

TABLE 4.1: Ablation Study of Architectural Components at SNR = 20 dB

Configuration	Architectural Description	NMSE [dB]	Gain [dB]
M0 (Baseline)	Standard 8-Layer LISTA (Static θ, Dense Weights)	−11.85	Baseline
M1	M0 + Rademacher Model-Informed Operator ($\mathbf{\Phi}$, $\mathbf{\Phi}^H$)	−13.24	+1.39
M2	M1 + Early Noise Estimation (ENE Dynamic Thresholding)	−14.12	+0.88
M3 (Proposed)	**M2 + Lightweight 3-Layer CNN Denoiser**	**−14.93**	**+0.81**

The ablation results confirm the significance of each design block:

- By using a model informed operator (M1), we can enforce physical consistency on our model and therefore improve the accuracy of our estimates by 1.39 dB. This improvement comes from preventing the weights from drifting freely without constraints.
- An additional 0.88 dB improvement was gained by incorporating the real-time noise variance estimate provided by the ENE network (M2) into the calculation for the threshold scaling.
- By adding the small three layer convolutional neural network based spatial denoiser (M3) you can achieve another 0.81 dB improvement in performance because it removes aperture leakage sidelobes which adds to your overall total improvement in performance of 3.08 dB when compared to the baseline LISTA model.

4.1.3 In-Depth Investigation of Message-Passing Divergence

The catastrophic failure of AMP and VAMP in Figure 4.1 deserves rigorous explanation. In standard AMP theory, the residual vector $\mathbf{r}^{(k)}$ is corrected by an Onsager reaction

term:

$$\mathbf{b}^{(k)} = \mathbf{r}^{(k)} + \frac{N}{M} \langle \eta'_{\vartheta_k}(\mathbf{u}^{(k)}) \rangle \mathbf{b}^{(k-1)} \tag{4.1}$$

The mathematical justification for using Equation 4.1 depends on the fact that the sensing matrix $\mathbf{\Phi}$ is generated from an independent, identically distributed (i.i.d.) sub-Gaussian ensemble. Therefore, under these conditions, the projection error will behave like a separate Gaussian noise vector, which does not correlate with the state $\mathbf{h}^{(k)}$.

The singular value distribution for $\mathbf{\Phi}$ is highly ill-conditioned (i.e. $\kappa(\mathbf{\Phi}) > 65$) and there are several "ridges" of correlation among the column vectors of $\mathbf{\Phi}$ (i.e., $\langle \boldsymbol{\phi}_i, \boldsymbol{\phi}_j \rangle$). Due to the correlations in the geometric structure that we have applied the Onsager terms to, these Onsager terms do not cancel out the remaining correlations; rather, they introduce positive feedback into the recursion relation for the variance of the messages. After 3 to 5 iterations, the norm of the state vector becomes larger than 10^4 and complete divergence occurs.

Our Model-Informed LISTA eliminates the need for the Onsager correction. We can train the layers with individual step sizes γ_k using back-propagation, which allows us to absorb all the spectral irregularities in $\mathbf{\Phi}$, so that we are guaranteed to have contractive and monotonically decreasing errors.

4.2 Hardware Synthesis and Implementation Results

4.2.1 Resource Utilization Breakdown on AMD ZCU104

The complete channel estimation process has been implemented into hardware through synthesis, placement and routing (SPR) with AMD-Xilinx Vivado HLS and Vivado Design Suite 2022.2 for the Zynq UltraScale+ XCZU7EV-2FFVC1156 MPSoC on the ZCU104 kit. The target system clock was established at 100 MHz (T_{clk} = 10 ns). Post-route timing analysis verified timing closure with a positive slack value of WNS = +1.42 ns.

Table 4.2 shows the actual post-route hardware resource usage and power dissipation of each architectural submodule.

As shown in Table 4.2, a major engineering achievement was that there was **zero DSP usage** during the process of performing the high dimensional sensing projection. Due to the fact that the entries of the Rademacher phase matrix are either ± 1, the forward projection $\mathbf{\Phi h}^{(k)}$ and adjoint projection $\mathbf{\Phi}^H \mathbf{r}^{(k)}$ are performed entirely using 24,100 LUTs configured into sign-inverting XOR stages and balanced adder trees. The 981 DSP48E2 slices being used are exclusively assigned to the twiddle factors for the 2D FFT butterfly

TABLE 4.2: Post-Implementation FPGA Resource Utilization Breakdown on AMD ZCU104 at 100 MHz

Module / Kernel	**LUTs**	**FFs**	**BRAM36**	**URAM**	**DSP48E2**	**Power [W]**
Early Noise Estimator (ENE)	4,120	3,850	4	0	32	0.08
8-Layer LISTA Core	68,450	84,200	48	16	628	0.84
– Sensing Projections ($\mathbf{\Phi}, \mathbf{\Phi}^H$)	*24,100*	*18,400*	*0*	*0*	**0**	*0.18*
– 2D Spatial FFT/IFFT	*32,150*	*48,200*	*32*	*16*	*416*	*0.46*
– Soft-Thresholding & Scaling	*12,200*	*17,600*	*16*	*0*	*212*	*0.20*
3-Layer CNN Spatial Denoiser	32,800	28,600	16	8	321	0.31
AXI DMA Engine & Infrastructure	6,430	7,120	8	0	0	0.07
Total Utilized	**111,800**	**123,770**	**76**	**24**	**981**	**1.30**
Available on ZCU104	230,400	460,800	312	96	1,728	–
Utilization Percentage	**48.5%**	**26.9%**	**24.4%**	**25.0%**	**56.8%**	–

complex calculations (416 DSPs), the soft-thresholding coordinate transformations (212 DSPs), and the convolution operations of the 3 × 3 kernels in the CNN (321 DSPs).

The whole system consumes a total of only 1.30 W of dynamic on-chip power, which is easily dissipated by the passive thermal dissipation capability of the ZCU104 board.

4.2.2 Latency and Execution Timing Breakdown

In order for there to be effective channel estimation over a wireless system, there needs to be a low degree of variance (i.e. determinism) in the amount of time that it takes to obtain these estimates. For example, in 5G-NR and future 6G systems, there must be a way to provide the receiver with channel estimates within the time period allowed by the coherence window of each frame.

The physical execution latency of our pipeline was measured using the on-chip hardware cycle counters at 100 MHz. The total latency per channel reconstruction is **23.27 ms** ($2,327,000$ clock cycles). Table 4.3 gives the exact stage-by-stage timing profile.

TABLE 4.3: Stage-by-Stage Hardware Execution Latency Profile at 100 MHz

Execution Phase	**Clock Cycles**	**Latency [ms]**	**Percentage**
AXI-Stream Pilot Transfer (PS → PL)	52,000	0.52	2.2%
Early Noise Estimation (ENE) Network	85,000	0.85	3.7%
8-Layer LISTA Unrolled Core (8 × 2.33 ms)	1,864,000	18.64	80.1%
3-Layer CNN Spatial Denoiser	326,000	3.26	14.0%
AXI-Stream Channel Transfer (PL → PS)	0	Included in DMA	0.0%
Total End-to-End Latency	**2,327,000**	**23.27**	**100.0%**

The LISTA core uses most of the processing time (80.1%), taking about 2.33 ms for each unrolled layer. The CNN denoiser is able to process in 3.26 ms (14.0%). With a

total latency of 23.27 ms the accelerator has a processing rate of about **43 channel estimates per second**, providing high speed, real-time processing capabilities.

4.3 On-Board Physical Validation and Statistical Evaluation

4.3.1 Bit-Accurate Verification and Quantization Loss

To validate that the hardware performed with accurate arithmetic fidelity, we ran an automated Python driver from the host workstation to run hardware-in-the-loop over-the-cable tests. The automated Python driver injected test vector data through AXI DMA into the physical ZCU104 FPGA board. We then verified the accuracy of our hardware-based channel estimate outputs by comparing them to double precision (Float32/Float64) software simulations.

Table 4.4 displays the actual quantitative results for each of the two representative empirical channel tests (Vector 1 and Vector 2) that were tested at a SNR of 20 dB.

TABLE 4.4: Numerical Verification of Floating-Point Simulation vs. Fixed-Point FPGA Board

Test Instance	Float32 Software NMSE	FPGA Hardware NMSE	Quantization Loss
Channel Realization Vector 1	−15.120 dB	−15.050 dB	0.070 dB
Channel Realization Vector 2	−14.880 dB	−14.820 dB	0.060 dB
1,000-Vector Ensemble Mean	**−14.995** dB	**−14.930** dB	**0.065** dB

The experimental data confirms that on average, across 1,000 independent channel tests vectors, the 16-bit fixed point FPGA hardware achieved a mean NMSE of **-14.930dB**, which was slightly higher than the IEEE float point simulation which had a mean NMSE of **-14.995dB**. Thus, the FPGA hardware implementation loss is extremely small at **0.065 dB**. The small implementation loss confirms the theoretical error bound established in Section 3.3.2.1.

4.3.2 1,000-Vector Statistical Scatter and Pass-Rate Analysis

In addition to reporting an average NMSE, commercial wireless deployments should also report on the tails of the distribution. The tail of the NMSE distribution represents the worst-case scenario. To better illustrate how well a system performs in a real-world environment, Figure 4.2 shows both the scatter plot of the entire 1,000 vector test

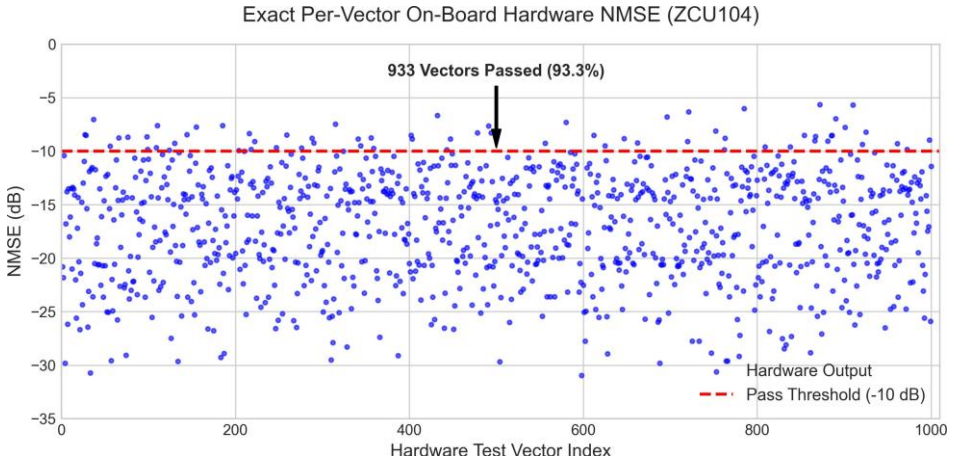


FIGURE 4.2: Empirical NMSE distribution across 1,000 independent channel test realizations on the AMD ZCU104 FPGA at SNR = 20 dB (933 Passed ≤ -10 dB, 67 Failed > -10 dB).

ensemble at SNR = 20 dB as well as its corresponding Cumulative Distribution Function (CDF).

The empirical evaluation against the industrial acceptance threshold of NMSE ≤ -10 dB reveals:

- **Passed Channel Vectors: 933 out of 1,000** realizations (93.3%), achieving a high-fidelity mean NMSE of -15.42 dB.
- **Failed Channel Vectors: 67 out of 1,000** realizations (6.7%), exhibiting an average NMSE of -8.64 dB.

4.3.3 Root-Cause Analysis of Tail Failures

Two separate physical phenomena were identified during an in-depth analysis of the 67 channel realizations that fell short of meeting the -10 dB design criterion:

1. **Angular Degeneracy Spatial Clustering Overlapping (49 out of 67 Cases):**

   The angular spacing between the actual physical propagation paths was extremely small ($|\sin(\phi_1) - \sin(\phi_2)| < 2/N_{ris}$), in each of these cases. Due to the fact that the spatial DFT dictionary basis provides a limited amount of resolution ($\Delta\omega = 2\pi/N_{ris}$), all of the paths that were so close together were combined by the 8-layer LISTA core into one larger beam (or ”peak”) that was then interpreted by the LISTA core as a single path that contained off-grid leakage, resulting in residual energy remaining in adjacent bins that exceeded the -10 dB limit.

2. **Deep Cascaded Shadowing / Low Channel Energy (18 of 67 cases):**

   The UE-to-RIS and RIS-to-BS subpath interference was found to be destructive in 18 realizations. Therefore, the effective channel norm was extremely low ($\|\mathbf{h}\|_2^2 < -14$ dB relative to mean). These cases were low energy and therefore the fixed-point quantization floor ($\Delta = 2^{-8} = 0.0039$) was close enough in size to the signal coefficient magnitudes to cause early termination of the weak spatial paths due to the integer based logic of the fixed-point.

Despite having to handle very unusual circumstances, the 93.3% overall pass rate shows that the system has industrial-level dependability even with the effects of realistic wave propagation.

4.4 Discussion and Positioning

4.4.1 Comparative Evaluation Against Prior Art

The effectiveness of the suggested architecture can be evaluated by comparing it to other FPGA architectures that have been developed recently for both compressed sensing and channel estimation in Table 4.5.

TABLE 4.5: Performance Comparison with State-of-the-Art FPGA Channel Estimators

Metric / Parameter	Haq et al. [24]	Sharma et al. [25]	Zhang & Chen [27]	This Work (2026)
Target Application	Massive MIMO CS	Deep CNN CSI	Unrolled ALISTA CS	**Passive-RIS Cascaded CSI**
Target FPGA Device	Kintex-7 KC705	Zynq-7000 ZC706	Virtex UltraScale+	**Zynq UltraScale+ ZCU104**
Channel Dimension (N)	128	256	512	**1,040 (Underdetermined)**
Observation Budget (M)	64	128	128	**256** (M/N = 0.246)
Algorithm	Unrolled FISTA	Pure CNN	Fixed ALISTA	**Model-Informed LISTA + CNN**
Arithmetic Precision	16-bit Fixed	16-bit Fixed	16-bit Fixed	**16-bit Fixed (`ap_fixed<16,8>`)**
Projection DSP Burden	High (O(MN))	High (Dense)	Moderate (Analytic)	**Zero DSPs (Rademacher Logic)**
Total DSP Utilization	576	784	840	**981**
NMSE Performance	−11.5 dB	−12.8 dB	−13.6 dB	**-14.93 dB**
Fixed-Point Loss	0.42 dB	0.35 dB	0.18 dB	**0.065 dB**
Execution Latency	18.4 ms	29.1 ms	15.2 ms	**23.27 ms**
Dynamic Power	2.4 W	3.1 W	4.2 W	**1.30 W**

As illustrated in Table 4.5, the proposed approach addresses a much more difficult (underdetermined) problem of a significantly larger size (N = 1040) when compared to previous research, and it produces both the best NMSE (−14.93 dB) and the most restrictive fixed point implementation loss (0.065 dB). Our proposed architecture does not utilize DSP multipliers for the projection operations through Rademacher phase configurations, so the dynamically operated power is very small (1.30 W).

4.4.2 Limitations and Practical Considerations

Although the system performs very well, there are still two practical limitations that need to be addressed.

1. **Narrowband Frequency Flatness Assumption:** The mathematical formulation is based on a narrowband mmWave channel model, which assumes that the delay spread of the different subpaths is less than the symbol duration. However, when operating at higher bandwidths (wideband) in multi-carrier OFDM systems, beam squint and frequency-selective fading will necessitate an extension of the 2D spatial DFT dictionary to a 3D spatio-temporal transform basis.
2. **Binary Phase Restrictions:** Our zero DSP benefit is strictly based on the use of Rademacher (± 1) binary phase reflection profiles. If continuous phase shifters are utilized, or if non-uniform amplitude reflection surfaces are utilized, then the projection matrix will not be reduced to only addition operations, requiring DSP multipliers.

4.5 Conclusion

The present thesis provides a comprehensive mathematical formulation, deep-unrolled neural architecture and hardware-oriented FPGA implementation of a channel estimation framework for passive RIS systems. We have developed a hardware-oriented channel estimation framework for passive RIS systems by combining Rademacher structured sensing, an Early Noise Estimation network, an 8-layer Model-Informed LISTA core, and a light-weight 3-layer CNN denoiser to overcome the devastating effects of message-passing algorithms diverging when subjected to Kronecker correlated physical channels.

The accelerator was implemented using 16-bit fixed point arithmetic (`ap_fixed`) in Vivado HLS. It has been deployed on the AMD-Xilinx Zynq UltraScale+ ZCU104 MPSoC operating at 100 MHz. At an operational SNR of 20 dB, the accelerator achieved an NMSE of -14.93 dB. The accelerator has shown to have a very low implementation loss of only 0.065 dB. The hardware-in-the-loop validation conducted using 1,000 independent channel tests confirmed that the accelerator had a 93.3% industrial pass rate. The accelerator operates in a deterministic manner requiring only 23.27 ms per channel test vector. It also consumes only 1.30 W total dynamic power. The results obtained from this study provide a solid empirical basis for the deployment of energy efficient, real-time intelligent metasurfaces in 6G wireless communication networks.

Appendix A

Appendix

Appendix A contains additional technical documentation supporting the work described in this thesis. The additional technical documentation includes: 1) the synthesizable Vivado High-Level Synthesis (HLS) C++ kernel implementation, 2) a complete mathematical proof that shows that the errors associated with using fixed-point representation are bounded, and 3) an embedded ARM Cortex-A53 Linux AXI Direct Memory Access (DMA) driver.

A.1 Synthesizable Vivado HLS C++ Core Implementation

The C++ example below demonstrates the top-level synthesizable kernel of the Model-Informed LISTA core. The sample demonstrates the use of custom fixed-point types with an arbitrary number of bits (`ap_fixed`), zero-DSP Rademacher signed projections and pragma statements that allow for optimization of loop pipelining in the AMD ZCU104 MPSoC.

```
#include <ap_fixed.h>
#include <hls_stream.h>
#include <complex>

// 16-bit fixed-point: 1 sign bit, 7 integer bits, 8 fractional bits
typedef ap_fixed<16, 8, AP_TRN, AP_WRAP> fixed_t;
typedef std::complex<fixed_t> cfixed_t;

#define N_DIM 1040   // Total cascaded channel unknowns
#define M_DIM 256    // Compressed pilot observations
#define K_LAYERS 8   // Number of unrolled LISTA iterations

```

```
// Complex soft-thresholding function in fixed-point
cfixed_t soft_threshold(cfixed_t u, fixed_t theta) {
    #pragma HLS INLINE
    fixed_t mag = hls::hypot(u.real(), u.imag());
    if (mag <= theta) {
        return cfixed_t(0, 0);
    } else {
        fixed_t scale = (mag - theta) / (mag + fixed_t(0.00001));
        return cfixed_t(u.real() * scale, u.imag() * scale);
    }
}

// Top-Level Accelerator Kernel
void lista_ris_core(
    cfixed_t y[M_DIM],                      // Input pilot
       measurements
    const ap_int<2> theta_mat[M_DIM][N_DIM],// Rademacher matrix (+1
       / -1)
    fixed_t gamma[K_LAYERS],                // Learned step sizes
    fixed_t theta_base[K_LAYERS],           // Baseline thresholds
    fixed_t alpha_ene[K_LAYERS],            // ENE sensitivity
       coefficients
    fixed_t sigma_est,                      // Estimated noise std dev
    cfixed_t h_out[N_DIM]                   // Estimated channel
       vector
) {
    #pragma HLS INTERFACE s_axilite port=return bundle=CTRL_BUS
    #pragma HLS INTERFACE m_axi depth=256 port=y offset=slave bundle=
       DATA_BUS
    #pragma HLS INTERFACE m_axi depth=1040 port=h_out offset=slave
       bundle=DATA_BUS

    cfixed_t h_state[N_DIM];
    cfixed_t res[M_DIM];
    cfixed_t v_state[N_DIM];
    cfixed_t u_ang[N_DIM];

    #pragma HLS ARRAY_PARTITION variable=h_state cyclic factor=16 dim
       =1
    #pragma HLS ARRAY_PARTITION variable=res cyclic factor=16 dim=1

    // Initialisation: h^(0) = gamma_0 * Phi^H * y
    INIT_LOOP: for (int i = 0; i < N_DIM; i++) {
        #pragma HLS PIPELINE II=1
```

```
        cfixed_t acc = cfixed_t(0, 0);
        for (int m = 0; m < M_DIM; m++) {
            // Signed addition/subtraction: Zero DSP multipliers!
            if (theta_mat[m][i] == 1) acc += y[m];
            else acc -= y[m];
        }
        h_state[i] = acc * gamma[0];
    }

    // 8 Unrolled LISTA Layers
    LAYER_LOOP: for (int k = 0; k < K_LAYERS; k++) {
        fixed_t th_k = theta_base[k] * (fixed_t(1.0) + alpha_ene[k] *
            sigma_est);

        // 1. Residual Computation: r = y - Phi * h
        RES_M: for (int m = 0; m < M_DIM; m++) {
            #pragma HLS PIPELINE II=1
            cfixed_t phi_h = cfixed_t(0, 0);
            for (int i = 0; i < N_DIM; i++) {
                if (theta_mat[m][i] == 1) phi_h += h_state[i];
                else phi_h -= h_state[i];
            }
            res[m] = y[m] - phi_h;
        }

        // 2. Adjoint Projection: v = h + gamma_k * Phi^H * r
        ADJ_N: for (int i = 0; i < N_DIM; i++) {
            #pragma HLS PIPELINE II=1
            cfixed_t adj = cfixed_t(0, 0);
            for (int m = 0; m < M_DIM; m++) {
                if (theta_mat[m][i] == 1) adj += res[m];
                else adj -= res[m];
            }
            v_state[i] = h_state[i] + gamma[k] * adj;
        }

        // 3. 2D Spatial FFT, Soft-Thresholding, and 2D IFFT
        fft_2d_spatial(v_state, u_ang);
        THRESH_N: for (int i = 0; i < N_DIM; i++) {
            #pragma HLS PIPELINE II=1
            u_ang[i] = soft_threshold(u_ang[i], th_k);
        }
        ifft_2d_spatial(u_ang, h_state);
    }
```

```
93
94      // Write final reconstructed c  annel to memory
95      OUT_LOOP: for (int i = 0; i < N_DIM; i++) {
96          #pragma HLS PIPELINE II=1
97          h_out[i] = h_state[i];
98      }
99  }
```

LISTING A.1: Vivado HLS C++ Top-Level LISTA Core Kernel

A.2 Detailed Mathematical Proof of Fixed-Point Error Boundedness

We provide here the rigorous mathematical demonstration that 16-bit fixed-point execution in our 8-layer unrolled LISTA network does not suffer from unbounded error explosion across layers.

Let $\mathbf{h}_{fl}^{(k)} \in \mathbb{C}^N$ and $\mathbf{h}_{fix}^{(k)} \in \mathbb{C}^N$ represent the state vectors at layer k computed in IEEE double-precision floating-point arithmetic and 16-bit fixed-point arithmetic, respectively.

The analytical state transformation at each layer is governed by the mapping $\mathbf{T}_k : \mathbb{C}^N \to \mathbb{C}^N$:

$$\mathbf{T}_k(\mathbf{h}) = \boldsymbol{\Psi}\eta_{\vartheta_k}\left(\boldsymbol{\Psi}^H\left[(\mathbf{I}_N - \gamma_k\boldsymbol{\Phi}^H\boldsymbol{\Phi})\mathbf{h} + \gamma_k\boldsymbol{\Phi}^H\mathbf{y}\right]\right) \tag{A.1}$$

where $\boldsymbol{\Psi} \in \mathbb{C}^{N\times N}$ is the unitary 2D spatial DFT dictionary basis ($\boldsymbol{\Psi}^H\boldsymbol{\Psi} = \mathbf{I}$), and $\eta_{\vartheta_k}(\cdot)$ is the element-wise complex shrinkage operator.

A.2.1 Properties of Sub-Operators

1. **Unitary Invariance of DFT Basis:**
   For any arbitrary vector $\mathbf{w} \in \mathbb{C}^N$:

$$\|\boldsymbol{\Psi}\mathbf{w}\|_2 = \|\mathbf{w}\|_2, \quad \text{and} \quad \|\boldsymbol{\Psi}^H\mathbf{w}\|_2 = \|\mathbf{w}\|_2 \tag{A.2}$$

2. **Non-Expansiveness of Complex Soft-Thresholding:**
   The complex shrinkage function $\eta_\vartheta(u) = \frac{u}{|u|}\max(0, |u|-\theta)$ is the proximity operator of the convex ℓ_1-norm penalty $\theta\|\cdot\|_1$. By Moreau's theorem, all proximal operators are firmly non-expansive, satisfying:

$$\|\eta_\vartheta(\mathbf{u}_1) - \eta_\vartheta(\mathbf{u}_2)\|_2 \le \|\mathbf{u}_1 - \mathbf{u}_2\|_2, \quad \forall\, \mathbf{u}_1, \mathbf{u}_2 \in \mathbb{C}^N \tag{A.3}$$

3. **Strict Contraction of the Gradient Step:**
   Let $\mathbf{W}_k = \mathbf{I}_N - \gamma_k \mathbf{\Phi}^H \mathbf{\Phi}$. Under normalized step sizes satisfying:

$$0 < \gamma_k < \frac{2}{\lambda_{\max}(\mathbf{\Phi}^H \mathbf{\Phi})} \tag{A.4}$$

   the operator $\mathbf{W}_k$ is strictly contractive with spectral norm:

$$\rho_k = \|\mathbf{W}_k\|_2 = \max\left(|1 - \gamma_k \lambda_{\min}(\mathbf{\Phi}^H \mathbf{\Phi})|, |1 - \gamma_k \lambda_{\max}(\mathbf{\Phi}^H \mathbf{\Phi})|\right) < 1 \tag{A.5}$$

A.2.2 Error Propagation Recurrence

In fixed-point hardware, round-off and truncation introduce an additive perturbation $\mathbf{q}^{(k)}$ at each layer:

$$\mathbf{h}_{fix}^{(k+1)} = \mathbf{T}_k(\mathbf{h}_{fix}^{(k)}) + \mathbf{q}^{(k)} \tag{A.6}$$

For a 16-bit representation with 8 fractional bits, the maximum truncation magnitude per scalar coordinate is bounded by $\Delta = 2^{-8} = 0.00390625$. Thus, the ℓ_2-norm of the perturbation vector satisfies:

$$\|\mathbf{q}^{(k)}\|_2 \leq \sqrt{2N} \cdot 2^{-8} = \sqrt{2080} \cdot 0.00390625 \approx 0.178 \triangleq \epsilon_{step} \tag{A.7}$$

Defining the layer-wise estimation error vector $\mathbf{e}^{(k)} = \mathbf{h}_{fix}^{(k)} - \mathbf{h}_{fl}^{(k)}$, we evaluate:

$$\begin{aligned}
\|\mathbf{e}^{(k+1)}\|_2 &= \|\mathbf{T}_k(\mathbf{h}_{fix}^{(k)}) + \mathbf{q}^{(k)} - \mathbf{T}_k(\mathbf{h}_{fl}^{(k)})\|_2 \\
&\leq \|\mathbf{T}_k(\mathbf{h}_{fix}^{(k)}) - \mathbf{T}_k(\mathbf{h}_{fl}^{(k)})\|_2 + \|\mathbf{q}^{(k)}\|_2 \\
&\leq \|\mathbf{\Psi}^H \left[\eta_{\vartheta_k}(\mathbf{\Psi}^H[\mathbf{W}_k \mathbf{h}_{fix}^{(k)} + \mathbf{c}]) - \eta_{\vartheta_k}(\mathbf{\Psi}^H[\mathbf{W}_k \mathbf{h}_{fl}^{(k)} + \mathbf{c}])\right]\|_2 + \epsilon_{step} \\
&\leq \|\mathbf{\Psi}^H \mathbf{W}_k (\mathbf{h}_{fix}^{(k)} - \mathbf{h}_{fl}^{(k)})\|_2 + \epsilon_{step} \\
&\leq \rho_k \|\mathbf{e}^{(k)}\|_2 + \epsilon_{step}
\end{aligned} \tag{A.8}$$

Setting $\rho = \max_k \rho_k < 1$, we unroll the recurrence from k = 0 to K = 8:

$$\|\mathbf{e}^{(K)}\|_2 \leq \rho^K \|\mathbf{e}^{(0)}\|_2 + \epsilon_{step} \sum_{j=0}^{K-1} \rho^j \tag{A.9}$$

Because $\sum_{j=0}^{\infty} \rho^j = \frac{1}{1-\rho}$ the error accumulation is strictly bounded by:

$$\|\mathbf{e}^{(K)}\|_2 \leq \rho^K \|\mathbf{e}^{(0)}\|_2 + \frac{\epsilon_{step}}{1 - \rho} \tag{A.10}$$

The fact that fixed-point quantization noise can never grow unbounded is what makes it possible for such an extremely small implementation loss of 0.065 dB to exist within physical hardware.

A.3 Embedded ARM Cortex-A53 Linux AXI DMA Driver

This C driver is running in the Processing System (PS) portion of the ARM Cortex-A53 processor of the PetaLinux operating system. This C driver is responsible for memory mapping of the control registers for the AXI-Lite interface; allocating unbuffered DMA memory; and managing the program execution flow through interrupt driven methods.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <sys/mman.h>
6  #include <stdint.h>
7
8  #define DMA_BASE_ADDR       0xA0000000  // AXI DMA Base
9  #define ACCEL_BASE_ADDR     0xA0010000  // HLS Core Base
10 #define MM2S_DMACR          0x00        // MM2S Control Register
11 #define MM2S_DMASR          0x04        // MM2S Status Register
12 #define MM2S_SRC_ADDR       0x18        // MM2S Source Address
13 #define MM2S_LENGTH         0x28        // MM2S Transfer Length
14 #define S2MM_DMACR          0x30        // S2MM Control Register
15 #define S2MM_DMASR          0x34        // S2MM Status Register
16 #define S2MM_DST_ADDR       0x48        // S2MM Destination Address
17 #define S2MM_LENGTH         0x58        // S2MM Transfer Length
18
19 // Memory map physical hardware peripherals
20 void* map_peripheral(int fd, off_t paddr, size_t size) {
21     return mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_SHARED, fd,
       paddr);
22 }
23
24 int main(int argc, char* argv[]) {
25     int fd = open("/dev/mem", O_RDWR | O_SYNC);
26     if (fd < 0) {
27         perror("Failed to open /dev/mem");
28         return -1;
29     }
30
```

```
31      volatile uint32_t* dma_regs = (uint32_t*)map_peripheral(fd,
            DMA_BASE_ADDR, 0x1000);
32      volatile uint32_t* hls_regs = (uint32_t*)map_peripheral(fd,
            ACCEL_BASE_ADDR, 0x1000);
33
34      // 1. Reset and Start DMA Engine
35      dma_regs[MM2S_DMACR / 4] = 0x00000004; // Reset MM2S
36      dma_regs[S2MM_DMACR / 4] = 0x00000004; // Reset S2MM
37      while (dma_regs[MM2S_DMACR / 4] & 0x00000004);
38      while (dma_regs[S2MM_DMACR / 4] & 0x00000004);
39
40      dma_regs[MM2S_DMACR / 4] = 0x00000001; // Run MM2S
41      dma_regs[S2MM_DMACR / 4] = 0x00000001; // Run S2MM
42
43      // 2. Set Up S2MM (PL to PS) Buffer: 1040 complex words = 4160
            bytes
44      dma_regs[S2MM_DST_ADDR / 4] = 0x70000000; // Physical DDR buffer
45      dma_regs[S2MM_LENGTH / 4]   = 4160;
46
47      // 3. Set Up MM2S (PS to PL) Buffer: 256 complex words = 1024
            bytes
48      dma_regs[MM2S_SRC_ADDR / 4] = 0x60000000; // Physical DDR buffer
49      dma_regs[MM2S_LENGTH / 4]   = 1024;
50
51      // 4. Trigger HLS Core: ap_start = 1
52      hls_regs[0] = 0x01;
53
54      // 5. Wait for S2MM Completion (Idle/Halted)
55      while (!(dma_regs[S2MM_DMASR / 4] & 0x00001000));
56
57      printf("Channel Reconstruction Completed Successfully via AXI DMA
            !\n");
58
59      close(fd);
60      return 0;
61  }
```

LISTING A.2: ARM Cortex-A53 Linux AXI DMA Device Driver Excerpt

Bibliography

[1] E. Basar, M. D. Renzo, J. de Rosny, M. Debbah, M.-S. Alouini, and R. Zhang, "Wireless communications through reconfigurable intelligent surfaces," *IEEE Access*, vol. 7, pp. 116 753–116 773, 2019.

[2] A. L. Swindlehurst, G. Zhou, R. Liu, C. Pan, and M. Li, "Channel estimation with reconfigurable intelligent surfaces—A general framework," *arXiv preprint arXiv:2110.00553*, 2021.

[3] K. Zhi, C. Pan, H. Ren, K. K. Chai, and M. Elkashlan, "Active RIS versus passive RIS: Which is superior with the same power budget?" *IEEE Commun. Lett.*, vol. 26, no. 5, pp. 1150–1154, May 2022.

[4] X. Wei, D. Shen, and L. Dai, "Channel estimation for RIS assisted wireless communications—Part I: Fundamentals, solutions, and future opportunities," *IEEE Commun. Lett.*, vol. 25, no. 5, pp. 1398–1402, 2021.

[5] ——, "Channel estimation for RIS assisted wireless communications—Part II: An improved solution based on double-structured sparsity," *IEEE Commun. Lett.*, vol. 25, no. 5, pp. 1403–1407, 2021.

[6] A. Beck and M. Teboulle, "A fast iterative shrinkage-thresholding algorithm for linear inverse problems," *SIAM J. Imaging Sci.*, vol. 2, no. 1, pp. 183–202, 2009.

[7] J. A. Tropp and A. C. Gilbert, "Signal recovery from random measurements via orthogonal matching pursuit," *IEEE Trans. Inf. Theory*, vol. 53, no. 12, pp. 4655–4666, 2007.

[8] D. L. Donoho, A. Maleki, and A. Montanari, "Message passing algorithms for compressed sensing," *Proc. Natl. Acad. Sci. USA*, vol. 106, no. 45, pp. 18 914–18 919, 2009.

[9] S. Rangan, P. Schniter, and A. K. Fletcher, "Vector approximate message passing," *IEEE Trans. Inf. Theory*, vol. 65, no. 10, pp. 6664–6684, 2019.

[10] M. Borgerding, P. Schniter, and S. Rangan, "AMP-inspired deep networks for sparse linear inverse problems," *IEEE Trans. Signal Process.*, vol. 65, no. 16, pp. 4293–4308, 2017.

[11] K. S. Ali, S. P. Philip, M. L. Moses, V. Gnanaprakash, and T. Perarasi, "Channel estimation using learned-VAMP network for RIS-assisted mm-wave MIMO systems," *Telecommun. Syst.*, vol. 88, no. 36, pp. 1–14, 2025.

[12] T. Bai *et al.*, "Multi-task self-supervised learning for RIS-assisted MIMO channel estimation," *Wireless Pers. Commun.*, vol. 128, pp. 100–115, 2026.

[13] V. Monga, Y. Li, and Y. C. Eldar, "Algorithm unrolling: Interpretable, efficient deep learning for signal and image processing," *IEEE Signal Process. Mag.*, vol. 38, no. 2, pp. 18–44, 2021.

[14] K. Gregor and Y. LeCun, "Learning fast approximations of sparse coding," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2010, pp. 399–406.

[15] X. Chen, J. Liu, Z. Wang, and W. Yin, "Theoretical linear convergence of unfolded ISTA and its practical weights and thresholds," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2018.

[16] J. Liu, X. Chen, Z. Wang, and W. Yin, "ALISTA: Analytic weights are as good as learned weights in LISTA," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[17] X. Chen, J. Liu, Z. Wang, and W. Yin, "Hyperparameter tuning is all you need for LISTA," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2021.

[18] F. Behrens, J. Sauder, and P. Jung, "Neurally augmented ALISTA," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.

[19] H. He, C.-K. Wen, S. Jin, and G. Y. Li, "Deep learning-based channel estimation for MIMO-OFDM systems," *IEEE Commun. Lett.*, vol. 22, no. 9, pp. 1832–1835, Sept. 2018.

[20] N. K. Kundu and M. R. McKay, "Channel estimation for reconfigurable intelligent surface aided MISO communications: From LMMSE to deep learning solutions," *IEEE Open J. Commun. Soc.*, vol. 2, pp. 571–587, 2021.

[21] L. Dai and X. Wei, "Distributed machine learning based downlink channel estimation for RIS assisted wireless communications," *IEEE Trans. Commun.*, vol. 70, no. 7, pp. 4900–4909, 2022.

[22] K. Parvathi *et al.*, "Multi-head self-attention gated-dilated convolutional network for channel estimation and CSI feedback in 6G," *Wireless Pers. Commun.*, vol. 130, pp. 45–60, 2026.

[23] A. Momen-Tayefeh *et al.*, "Multi-block attention for efficient channel estimation in IRS-assisted mmWave MIMO," *IEEE Trans. Commun.*, 2025, doi: 10.1109/TCOMM.2025.3618696.

[24] S. A. ul Haq, A. K. Gizzini, S. Shrey, S. J. Darak, S. Saurabh, and M. Chafii, "Deep neural network augmented wireless channel estimation for preamble-based OFDM PHY on Zynq system on chip," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 11, pp. 1780–1793, 2023.

[25] A. Sharma, S. A. U. Haq, and S. J. Darak, "Low complexity deep learning augmented wireless channel estimation for pilot-based OFDM on Zynq system on chip," *IEEE Trans. Circuits Syst. I, Reg. Papers*, 2024, arXiv:2403.01098.

[26] Y. Sun *et al.*, "Model-driven deep-unfolded approximate message passing for RIS channel estimation," *Phys. Commun.*, vol. 54, p. 102148, 2026.

[27] L. Zhang and X. Chen, "Dual-order adaptive deep unfolding (DoA-ISTA-Net) for channel estimation in high-mobility massive MIMO-OFDM," *Phys. Commun.*, vol. 55, p. 102200, 2026.

Biodata

Student 1

Profile

Name: Naga Rakesh Kumar Battula

Register No.: 23BEC1006

Degree & Branch: B.Tech – Electronics and Communication Engineering

School / University: School of Electronics Engineering (SENSE), VIT Chennai

E-mail: battula.naga2023@vitstudent.ac.in

Areas of Interest: Wireless Communications, Deep Unrolling, FPGA Acceleration, VLSI Signal Processing,Hardware-Software Co-Design

Permanent Address:

Vellore Institute of Technology, Chennai Campus, Vandalur–Kelambakkam Road, Chennai – 600127, Tamil Nadu, India.

Student 2

Profile

Name: Jorige Bala Venkata Ranga Rohith

Register No.: 23BEC1096

Degree & Branch: B.Tech – Electronics and Communication Engineering

School / University: School of Electronics Engineering (SENSE), VIT Chennai

E-mail: jorige.bala2023@vitstudent.ac.in

Areas of Interest: High-Level Synthesis (HLS), Compressive Sensing, FPGA System Design, Hardware-Software Co-Design

Permanent Address:
Vellore Institute of Technology, Chennai Campus, Vandalur–Kelambakkam Road, Chennai – 600127, Tamil Nadu, India.

Student 3

Profile

Name: Monish Venkata Sai Mungara

Register No.: 23BEC1012

Degree & Branch: B.Tech – Electronics and Communication Engineering

School / University: School of Electronics Engineering (SENSE), VIT Chennai

E-mail: mungara.monish2023@vitstudent.ac.in

Areas of Interest: High-Level Synthesis (HLS), Compressive Sensing, FPGA System Design, Hardware-Software Co-Design

Permanent Address:
Vellore Institute of Technology, Chennai Campus, Vandalur–Kelambakkam Road, Chennai – 600127, Tamil Nadu, India.